



Explaining quantum circuits with Shapley values: towards explainable quantum machine learning

Raoul Heese¹ · Thore Gerlach² · Sascha Mücke³ · Sabine Müller¹ · Matthias Jakobs³ · Nico Piatkowski²

Received: 12 August 2023 / Accepted: 6 February 2025
© The Author(s) 2025

Abstract

Methods of artificial intelligence (AI) and especially machine learning (ML) have been growing ever more complex, and at the same time have more and more impact on people's lives. This leads to explainable AI (XAI) manifesting itself as an important research field that helps humans to better comprehend ML systems. In parallel, quantum machine learning (QML) is emerging with the ongoing improvement of quantum computing hardware combined with its increasing availability via cloud services. QML enables quantum-enhanced ML in which quantum mechanics is exploited to facilitate ML tasks, typically in the form of quantum-classical hybrid algorithms that combine quantum and classical resources. Quantum gates constitute the building blocks of gate-based quantum hardware and form circuits that can be used for quantum computations. For QML applications, quantum circuits are typically parameterized and their parameters are optimized classically such that a suitably defined objective function is minimized. Inspired by XAI, we raise the question of the explainability of such circuits by quantifying the importance of (groups of) gates for specific goals. To this end, we apply the well-established concept of Shapley values. The resulting attributions can be interpreted as explanations for why a specific circuit works well for a given task, improving the understanding of how to construct parameterized (or variational) quantum circuits, and fostering their human interpretability in general. An experimental evaluation on simulators and two superconducting quantum hardware devices demonstrates the benefits of the proposed framework for classification, generative modeling, transpilation, and optimization. Furthermore, our results shed some light on the role of specific gates in popular QML approaches.

Keywords Quantum machine learning · Explainable machine learning · Shapley values

1 Introduction

Machine learning (ML) has a significant impact on many applications in different domains. With the ongoing improvement of ML models and their growing complexity, another aspect is becoming increasingly important in addition to pure performance: the explainability of ML systems (Mol-

nar 2022). Explainability¹ refers to methods that make the behavior of ML systems—or, more generally, artificial intelligence (AI) systems (Mukhamediev et al. 2022)—comprehensible for humans. Realizing explainable AI (XAI) is a highly non-trivial task with a potentially great impact on many applications and can therefore be considered as an important research field. For example, XAI can be used to analyze the fairness of models to avoid discrimination, or their security against adversarial attacks, to name just two important aspects. Currently, reasoning about decisions is typically only possible to a very limited extent or even intractable for state-of-the-art ML models. A review of XAI exceeds the scope of this paper and we instead refer to, e.g., (Marcinkevičs et al. 2020; Linardatos et al. 2020; Minh et al.

✉ Raoul Heese
raoul.heese@gmail.com

¹ Fraunhofer ITWM, Fraunhofer-Platz 1, Kaiserslautern 67663, Germany

² Fraunhofer IAIS, Schloss Birlinghoven 1, Sankt Augustin 53757, Germany

³ TU Dortmund, August-Schmidt-Straße 1, Dortmund 44227, Germany

¹ We do not distinguish between *explainability* and *interpretability* because these terms are not strictly defined and are often used interchangeably in the ML community (Marcinkevičs et al. 2020).

2022; Molnar 2022; Saeed and Omlin 2023) and references therein.

Apart from explainability, another promising research direction of ML that has recently arisen with the emergence of new technology is quantum machine learning (QML) (Biamonte et al. 2017; Maria Schuld 2018). This field addresses how quantum computers (or quantum information processing in general Nielsen et al. 2011) can be used to provide a benefit for ML. To identify possible advantages, the structure of the data seems to be key (Huang et al. 2021; Liu et al. 2021; Caro et al. 2022). However, since currently, only noisy intermediate-scale quantum (NISQ) devices (Preskill 2018) with limited capabilities are available, a practical application of QML is restricted to toy examples. On the other hand, the identification of quantum advantage is not necessarily of central importance for current research (Schuld and Killoren 2022). Instead, the study of fundamental aspects of QML and their prospective exploitation with better hardware in the future might be more promising.

With this in mind, it seems natural to study explainability for QML in a similar way as for *classical* (i.e., non-quantum) ML, giving rise to *explainable QML* (XQML) with the goal to provide humanly understandable interpretations of QML systems. This aspect of QML has been virtually unexplored at the time of the initial submission of this manuscript, except for one other study in the context of malware detection (Mercaldo et al. 2022). In the meantime, however, there have been a number of new research efforts that indicate a growing awareness of the topic. For example, in the form of review articles (Power and Guha 2024; Gujju et al. 2024) that also introduce novel methods (Gil-Fuster et al. 2024) as well as articles focusing on quantum neural networks (Ciaramella et al. 2023; Tian and Yang 2024; Pira and Ferrie 2024), the visualization of model behavior (Ruan et al. 2024; Lin et al. 2024), or reinforcement learning (Cuéllar et al. 2024; Flamini et al. 2024), just to name a few.

XAI methods can be divided into model-specific approaches and model-agnostic approaches. While the former are limited to specific model classes, the latter can be used for arbitrary models. Hence, model-agnostic XAI methods can be applied to QML models in the same way as to classical ML models. This leaves us with a rich set of classical tools that can be directly explored and potentially refined for XQML (Power and Guha 2024; Gil-Fuster et al. 2024). Since QML is still in a very early stage in comparison to ML, explainability can be adopted from the beginning for comparatively simple tasks that are much easier to analyze than the classical state-of-the-art problems. Their exploration could lead to fundamental insights, similar to the classical analog.

In the study of the decision-making process of classical ML systems, two central, interrelated questions are typically addressed by XAI:

1. What is the influence of the model inputs (i.e., features) on its predictions?
2. How does the model architecture (its components and/or trainable parameters) affect its predictions?

The first question takes an external perspective by investigating the (potentially black-box) relationship between inputs and outputs and can be considered the more commonly asked. The second shifts attention inward, exploring how the architecture and parameters of the model itself affect its behavior.

In the present paper, we limit ourselves to the second perspective and propose a framework for approaching model architecture investigations in an XQML context. Specifically, our framework allows to explore the influence of gates (or groups of gates) as the building blocks of gate-based quantum computations using Shapley value (SVs) (Shapley 1953; Aumann and Shapley 1974), a well-known model-agnostic tool in XAI (Strumbelj and Kononenko 2010; Lundberg and Lee 2017; Sundararajan et al. 2017). While our considerations are generally applicable to any kind of gate-based quantum computation, we focus mostly on QML applications driven by our original motivation for such an analysis. We refer to our approach as *Shapley values for quantum circuit explanations* (SVQXs).

The study of internal structures of ML models is an important topic in deep learning, for example, to visualize the behavior of complex systems (Zhang and Zhu 2018). Often, it is closely related to the search for an optimal model design, especially in the context of neural networks, where it is known as *neural architecture search* (NAS) (Salehin et al. 2024; Salmani Pour Avval et al. 2025) and can also be based on SVs (Leon 2014; Stier et al. 2018; Xiao et al. 2022; Diaz-Ortiz Jr et al. 2023; Oloulade et al. 2025). NAS is a special branch of automatic machine learning (AutoML), a research field concerned with the automation of the entire ML pipeline (Barbudo et al. 2023). This concept has also been transferred to QML, where it is known as *automatic quantum machine learning* (AutoQML) (Klau et al. 2023; Rybotycki and Gawron 2024). In this connection, several approaches have been proposed to optimize quantum circuit designs for QML applications in the sense of a *quantum circuit architecture search* (QAS) (Ding and Spector 2022; Du et al. 2022; Franken et al. 2022; Giovagnoli et al. 2023; Chen 2024). While our proposed SVQXs can in principle also be used in the context of QAS, we focus on the explainability aspect in this paper and demonstrate its potential for circuit design optimization only by way of example.

Our main contributions can be listed as follows:

- We present SVQXs as a method to measure the impact of every gate in a (variational) quantum circuit on the overall

expressibility, entanglement capability, or any arbitrary quantity of interest.

- The proposed method is grounded in the theory of uncertain SVs and therefore extends this domain by a quantum approach while facilitating inherent robustness to the noise from NISQ devices.
- SVQXs can be leveraged for XQML, which is the primary focus in this work.
- We perform experiments on simulators and actual quantum hardware to showcase different use cases.
- A software toolbox and the data from our experiments are publicly available online.

The remainder of this manuscript is divided into four sections. In Sect. 2, we introduce the prerequisites for our proposed method, which we present in Sect. 3. We subsequently study experimentally in Sect. 4 how our method can be used for a variety of applications. Finally, we conclude with a summary and outlook in Sect. 5.

2 Background

In the present section, we outline the prerequisites for our proposed XQML method of SVQXs. For this purpose, we first present a short summary of SVs and how they can be applied to classical ML. Next, we give a brief introduction to QML.

2.1 Shapley values

SVs (Shapley 1953; Aumann and Shapley 1974) are a concept originating in game theory that can be used to evaluate the performance of individual players in a coalition game. By definition, they fulfill many desirable properties that distinguish them as a particularly suitable evaluation measure for this purpose. Presume a group of $N \in \mathbb{N}$ players denoted by the set $\mathfrak{S} := \{1, \dots, N\}$, a coalition game is defined by the value function (or characteristic function)

$$v : \mathcal{P}(\mathfrak{S}) \rightarrow \mathbb{R}, \quad (1)$$

that assigns a (real-valued) payoff $v(S)$ to each subset (or *coalition*) of players $S \subseteq \mathfrak{S}$. The SV of player $i \in \mathfrak{S}$ is then given by the expectation value

$$\phi_i := \mathbb{E}[V_i] \quad (2)$$

of the random variable² $V_i \sim \mathbb{V}_i$ with probability mass function

$$\mathbb{V}_i(v) := \sum_{S \subseteq \mathfrak{S} \setminus \{i\}} w(S) \cdot \mathbb{I}(i, S, v). \quad (3)$$

² All random variables are in bold.

Here, we have introduced the weight

$$w(S) := \frac{1}{N} \binom{N-1}{|S|}^{-1} = \frac{|S|! (N - |S| - 1)!}{N!} \quad (4)$$

of coalition S based on its set cardinality $|S|$. Furthermore,

$$\mathbb{I}(i, S, v) := \begin{cases} 1, & \text{if } \Delta_i v(S) = v \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

denotes the indicator function with the marginal contribution

$$\Delta_i v(S) := v(S \cup \{i\}) - v(S) \quad (6)$$

of player i to the coalition S . In total, there are 2^N different coalitions including the grand coalition $\mathfrak{S}$ and the empty coalition $\emptyset$.

The SV framework is very general with typical applications in economy and finance (Aumann 1994; Winter 2002) as well as XAI (Strumbelj and Kononenko 2010). Due to the computational complexity of exact calculations, random sampling strategies can be employed to obtain approximate results (Fatima et al. 2007). A quantum algorithm for the calculation of (classical) SVs in an XAI context was recently proposed in Burge et al. (2024).

In our subsequent analyses, we apply SVs to quantum circuits. For this purpose, gates represent players and the value function is obtained from measurement results, for example, to quantify the contribution of each gate to produce a desired distribution of bits. Due to the universality of the theory, the value function can be chosen in many different ways depending on the specific use case. Our approach is explained in more detail in Sect. 3, where we also give detailed examples for possible value functions.

2.2 Shapley values under uncertainty

So far, we have considered a deterministic value function $v(S)$, Eq. 7. However, for value functions which are based on quantum processing unit (QPU) calculations, such a deterministic behavior is not given since results from QPUs are subject to

- The statistical uncertainty of measurements that results from the inherent uncertainty of quantum physics and the limited number of shots,
- The typically non-deterministic mapping of logical qubits onto physical qubits together with a decomposition of the circuit unitary that can be realized on the hardware in form of elementary gates (i.e., transpilation), and
- The hardware-related noise of NISQ devices as a result of, e.g., decoherence and dissipation effects (Burnett

et al. 2019) as well as state preparation and measurement (SPAM) errors (Sun and Geller 2018).

Consequently, in addition to randomness that occurs in classical ML algorithms (e.g., dropout Srivastava et al. 2014), additional randomness may arise in the context of QML. Handling value functions under randomness, noise or uncertainty is therefore essential for deriving meaningful SVs for QML. Recently, a theoretical framework for such value functions has been proposed (Heese et al. 2023). Following this reference, we consider an uncertain value function of the form

$$v(S) := v(S) + \mathbf{v}(S) \quad (7)$$

based on $v(S)$ from Eq. 1 with an additive noise given by the random variable $\mathbf{v}(S) \sim \mathbb{Q}(\cdot | S)$ that follows an arbitrary but fixed probability density $q(\cdot | S)$ for all $S \subseteq \mathcal{S}$. Here, $\mathbf{v}(S)$ is assumed to cover all sorts of noise which arise from QPU calculations. This form of additive noise implies a specific structure of the uncertainty under consideration. However, since the additive noise term in Eq. 7 explicitly depends on the coalition S , we can in fact use it to formally describe any kind of noisy behavior. As also already pointed out in Heese et al. (2023), it remains an open research question to evaluate other types of noise representations. In the following, we limit ourselves to the additive form. SVs for such uncertain value functions can then be defined in analogy to Eq. 8 as an expectation value

$$\Phi_i := \mathbb{E}[U_i] \quad (8)$$

of the random variable $U_i \sim \mathbb{P}_i$ with probability density

$$p_i(u) = \sum_{S \subseteq \mathcal{S} \setminus \{i\}} w(S) \int_{\mathbb{R}} q(v | S \cup \{i\}) q(v + \Delta_i v(S) - u | S) dv, \quad (9)$$

where we recall Eqs. 4 and 6. It can be shown that SVs of uncertain value functions as defined in Eq. 8 represent *regular* SVs in the sense of Eq. 2 with a deterministic but shifted value function. Therefore, all properties of regular SVs also apply for SVs of uncertain value functions. We refer to Heese et al. (2023) and references therein for a more detailed discussion.

In practice, the probability density $q(\cdot | S)$ is typically unknown (e.g., because the statistics of the noise are unknown) and Φ_i can consequently only be estimated. In the following, we briefly present two possible estimators, which are also outlined in Heese et al. (2023). For the first approach, the value function is sampled $K \in \mathbb{N}$ times for every coalition $S \in \mathcal{S}$. An unbiased estimator for Φ_i is then given by the sample mean

$$\hat{\Phi}_i^K := \frac{1}{K} \sum_{S \subseteq \mathcal{S} \setminus \{i\}} w(S) \sum_{k=1}^K [\tilde{v}^k(S \cup \{i\}) - \tilde{v}^k(S)], \quad (10)$$

where $\tilde{v}^k(S)$ represents the k th realization of the value function $v(S) \sim \mathbb{Q}(\cdot - v(S) | S)$ for $k \in \{1, \dots, K\}$. In total, this approach requires $2^N K$ value function samples. Due to its linearity, Eq. 10 can also be understood as a mean of K SVs, each based on one value function sample.

An alternative estimator can be employed without iterating K times over all 2^N possible coalitions. For this purpose, the coalitions themselves are treated as a random variable $S \sim w$, Eq. 4, and $n \in \mathbb{N}$ random samples are drawn from the set $\mathcal{S} \setminus \{i\}$ for each player i , which leads to the multiset of realizations $\mathcal{S}_i^n$. For each realization $S \in \mathcal{S}_i^n$, the value functions $v(S) \sim \mathbb{Q}(\cdot - v(S) | S)$ and $v(S \cup \{i\}) \sim \mathbb{Q}(\cdot - v(S) | S \cup \{i\})$ are both sampled K times. In total, $2nK$ value functions are sampled for each player i . Once the sampling for all N players is completed, all value function samples are collected in the multigroup $\mathcal{V}$. Thus, a (possibly empty) multiset of realizations $\mathcal{V}^i(S) := \{\tilde{v}^1(S), \dots, \tilde{v}^{|\mathcal{V}^i(S)|}(S)\}$ can be assigned to every coalition $S \in \mathcal{S}$ from the total collection $\mathcal{V}$. The latter corresponds to the additive union $\mathcal{V} = \biguplus_{S \in \mathcal{S}} \mathcal{V}(S)$. Similar to Eq. 10, $\tilde{v}^k(S)$ represents the k th realization of the value function for $k \in \{1, \dots, |\mathcal{V}(S)|\}$, where $|\mathcal{V}(S)| \geq 0$ denotes the number of value function samples for the coalition $S \in \mathcal{V}$. In conclusion, the expression

$$\hat{\Phi}_i^{n,K} := \frac{1}{n} \sum_{S \in \mathcal{S}_i^n} \left[\frac{1}{|\mathcal{V}(S \cup \{i\})|} \sum_{v \in \mathcal{V}(S \cup \{i\})} v + - \frac{1}{|\mathcal{V}(S)|} \sum_{v' \in \mathcal{V}(S)} v' \right] \quad (11)$$

also represents an unbiased estimator for Φ_i . In total, it requires up to $2nNK$ value function samples. This estimator can therefore be evaluated with less value function samples than Eq. 10, if n is chosen such that $n < 2^{N-1}/N$ (presuming that K is chosen equally).

SVs are defined as expectation values of random variables. The corresponding higher moments consequently allow further insight into the probability distribution of these random variables. For example, the standard deviation

$$\sigma[\Phi_i] := \sqrt{\mathbb{E}[U_i^2] - \Phi_i^2} \quad (12)$$

can be considered as a measure of the effective uncertainty of Φ_i .

2.3 Shapley values for classical machine learning

In order to lay out how SVs can be applied to quantum circuits, we recall their usage in the context of ML (Rozemberczki et al. 2022). We specifically limit ourselves to two types of applications that are related to our work: (i) SVs as

a model-agnostic XAI tool to explain the influence of model inputs on predictions and (ii) SVs as part of a NAS framework with the purpose to analyze the importance of the building blocks of neural networks. We only provide a brief summary of these two applications here, a detailed review of SVs for ML is beyond the scope of this paper.

For the first application, the use of SVs as a model-agnostic XAI tool (Strumbelj and Kononenko 2010; Lundberg and Lee 2017; Sundararajan et al. 2017; Stier et al. 2018; Xiao et al. 2022; Oloulade et al. 2025), features (i.e., distinct dimensions of a data point used as the model input) take the role of players and the value function is based on the corresponding model output. SVs of such a coalition game can then be used as an importance measure of each feature. This can be considered as the “traditional” usage of SVs in XAI.

Explanations are typically generated for a single data point, consisting of an N -dimensional feature vector $\vec{x} \in \mathbb{R}^N$. In this case, each feature $i \in \{1, \dots, N\}$ corresponds to a player and the value function is of the form

$$v(S) = v(S; \vec{x}, f), \quad (13)$$

where f denotes the model of interest. An alternative approach is to aggregate a score over multiple or all data points, which leads to feature importances with respect to an entire data set $\{\vec{x}_1, \dots, \vec{x}_d\}$ consisting of data points $\vec{x}_i \in \mathbb{R}^N$ for $i \in \{1, \dots, d\}$. Consequently, the value function is of the form

$$v(S) = v(S; \{\vec{x}_1, \dots, \vec{x}_d\}, f) \quad (14)$$

in analogy to Eq. 13.

In general, value functions of the form presented in Eqs. 13 and 14 can be realized in many different variants leading to SVs with different interpretations (Sundararajan et al. 2020). To evaluate coalitions of features, a common approach is to eliminate all features not present in a coalition S , introduced as SHAP by Lundberg and Lee (2017). To this end, the expectation of the model output is taken w.r.t. all features not present in the coalition, $\mathbb{E}[f(X) \mid x_S]$. Since computational complexity is a major issue for practical applications (Van den Broeck et al. 2022), this expectation is usually approximated by sampling from a given data set. Additional assumptions like feature independence and linearity can reduce the computational cost further. Specialized methods, e.g., KernelSHAP (Lundberg and Lee 2017), employ locally linear surrogate models to approximate SVs. There are also critical aspects of SVs in this context as for example studied in Kumar et al. (2020); Huang and Marques-Silva (2023).

For the second application, the use of SVs as part of a NAS framework (Leon 2014; Stier et al. 2018; Xiao et al. 2022; Diaz-Ortiz Jr et al. 2023; Oloulade et al. 2025), building

blocks of a neural network constitute the players, and the value function is based on the performance of the pruned network, which contains only the building blocks from the respective coalition.

Since (game theoretic) NAS can be implemented in many different ways (Stier et al. 2018), we present here a high-level description that focuses on the key ideas. A typical approach is to choose a specific ML task (e.g., image classification on a particular data set) and an appropriate neural network to solve this task. Then, the network is decomposed into c mutually excluding structural components (e.g., layers or parts of layers). Such a decomposition is not unique and each choice leads to a different coalition game. Formally, each structural component $i \in \{1, \dots, c\}$ corresponds to a player and the value function is of the form

$$v(S) = v(S; f, t), \quad (15)$$

where f denotes the neural network of interest and t the ML task (including the data). To evaluate Eq. 15, f is first pruned to contain only the structural elements that represent the players in the coalition S . Then, the pruned model is trained on the task t and a predefined performance metric (e.g., classification accuracy) is determined as the outcome of $v(S)$.

Within the NAS framework, the resulting SVs can then for example be used to iteratively prune the network by removing the structural components that are associated with the smallest SVs (Leon 2014). As for the first XAI application of explaining features, computational complexity is again a major issue for the practical realization of a SV-based NAS. A common solution is to use a random sampling approximation (Fatima et al. 2007).

2.4 Quantum machine learning

The interdisciplinary field of QML lies at the interface of ML and quantum computing (Biamonte et al. 2017; Maria Schuld 2018). It encompasses several research directions that consider data and algorithms that may both be classical, quantum, or a hybrid combination thereof. In this manuscript, we focus on quantum-enhanced ML (Dunjko et al. 2016), which considers the use of quantum algorithms to improve upon ML tasks for classical data. This is typically achieved by hybrid algorithms that include a classical and a quantum part. Such kind of algorithms can already be run on current NISQ devices.

In Fig. 1, we show a simplified sketch of such a *hybrid quantum-classical ML pipeline*, which we also shortly refer to as *hybrid ML pipeline* in the following. Classical data is presented to a classical *host*—typically consisting of (one or more) central processing unit (CPUs), graphics processing unit (GPUs), and RAM—which communicates with a QPU.

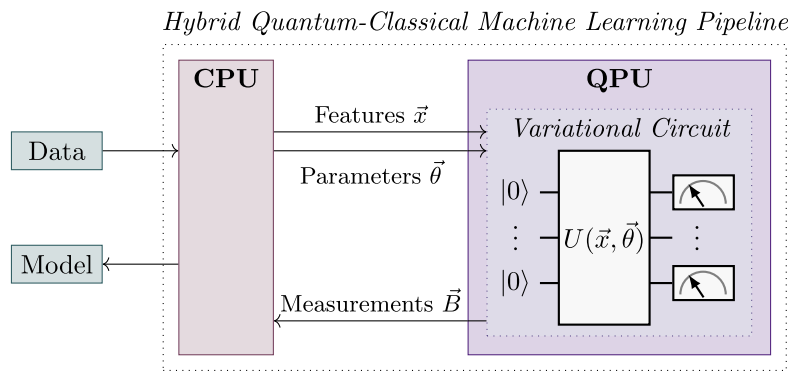


Fig. 1 Simplified sketch of a *hybrid quantum-classical ML pipeline* (or *hybrid ML pipeline* for short) in the form of a variational quantum algorithm. Preprocessed features $\vec{x}$ and parameters $\vec{\theta}$ from a classical host (represented here by a CPU) control a variational circuit that is executed on a QPU. The measurement results $\vec{B}$ are returned to the classical host

(in the form of bit strings) and enable a quantum-classical optimization loop for the parameters. In the end, a suitable model for the proposed ML problem can be realized. Prospectively, such a pipeline involves multiple CPUs and QPUs as well as GPUs (Gambetta 2022)

A quantum circuit is executed on the QPU. This circuit is controlled by features and variational parameters from the classical host. Features in this context refer to appropriately preprocessed classical data points that can be encoded in the circuit by a suitable combination of gates such that the resulting quantum state contains the respective information, for example in the form of amplitudes or angles (Weigold et al. 2020; Schuld et al. 2021). We denote a classical k -dimensional feature vector by $\vec{x} \in \mathcal{X} \subseteq \mathbb{R}^k$.

Furthermore, the variational parameters represent the degrees of freedom of the circuit depending on the chosen ansatz. An ansatz in this sense describes a particular sequence of gates, where each gate can depend on one or more variational parameters. We denote a p -dimensional vector of variational parameters by $\vec{\theta} \in \mathcal{P} \subseteq \mathbb{R}^p$. Quantum circuits depending on such parameters are also known as parameterized or *variational circuits* and are fully defined by a unitary operator $U(\vec{x}, \vec{\theta})$ acting on the joint Hilbert space of all q qubits that have initially been prepared in the ground state $|0\rangle$ as shown in Fig. 1. Hence, the unitary operator yields a final quantum state

$$|\Psi(\vec{x}, \vec{\theta})\rangle := U(\vec{x}, \vec{\theta})|0\rangle \quad (16)$$

of the joint system of qubits.

After execution, the measurement results are returned to the classical host. For s shots (or repeated circuit evaluations per execution), the measurement results consist of a multiset of s bit strings

$$\vec{B}(\vec{x}, \vec{\theta}) \equiv \vec{B} := \{\vec{b}_1, \dots, \vec{b}_s\}, \quad (17)$$

each of which represents the measured states of all q qubits in form of a vector $\vec{b}_i \in \{0, 1\}^q \forall i \in \{1, \dots, s\}$. In this

sense, each bit string $\vec{b}_i$ can be considered as a realization of a random variable $\vec{b}$ with probability mass

$$\mathbb{P}_{\vec{x}, \vec{\theta}}(\vec{b}) := \langle \Psi(\vec{x}, \vec{\theta}) | M(\vec{b}) | \Psi(\vec{x}, \vec{\theta}) \rangle, \quad (18)$$

where $M(\vec{b})$ represents the positive operator-valued measure (POVM) element of the measurement corresponding to the outcome $\vec{b}$ (Nielsen et al. 2011). Summarized, the variational quantum circuit can be understood as a mapping

$$\vec{x}, \vec{\theta} \mapsto \vec{B} \quad (19)$$

from features $\vec{x}$ and variational parameters $\vec{\theta}$ to a realization $\vec{B}$ of a random variable $\vec{B}$.

Typically, the variational parameters are optimized with a classical optimizer during training such that a suitable model for the proposed ML problem can eventually be realized after a certain number of quantum-classical iterations. Since the variational circuit is of central importance, this kind of hybrid ML pipeline is also referred to as variational quantum algorithm (VQA) (Cerezo et al. 2021). The resulting model involves a classical host and (optionally) a QPU for inference.

Classical ML and QML problems can be divided into three different categories:

- *Supervised learning.* Given a labeled dataset, the goal is to infer the label for unknown samples (see, e.g., Schuld 2021).
- *Unsupervised learning.* The task is to learn patterns of unlabeled data, for example, to perform clustering (see, e.g., Kyriienko and Magnusson 2022).
- *Reinforcement learning.* An agent is trained to learn actions that maximize a given reward function (see, e.g., Meyer et al. 2022).

All three problem categories can be approached with the aforementioned hybrid ML pipeline based on variational circuits.

For this purpose, measurement results are mapped onto a result value

$$h : \vec{B} \mapsto \vec{y} \in \mathcal{Y} \quad (20)$$

in some solution space $\mathcal{Y}$. For classification or clustering, $\mathcal{Y}$ may represent a set of class labels, whereas for regression, it can correspond to (a subset of) $\mathbb{R}^t$ for t -dimensional targets. For reinforcement learning, $\mathcal{Y}$ may be a set of feasible policies for an agent.

In each of those ML scenarios, there are common loss functions (or objectives) which are used to adjust the variational parameters $\vec{\theta}$ during training, e.g., through gradient descent or evolutionary optimization. Typically, a loss function

$$\ell : \{(\vec{x}_1, \vec{y}_1), \dots, (\vec{x}_d, \vec{y}_d)\}, \vartheta \mapsto \mathbb{R} \quad (21)$$

is aggregated over a training dataset $\{(\vec{x}_1, \vec{y}_1), \dots, (\vec{x}_d, \vec{y}_d)\}$ consisting of d tuples of feature vectors $\vec{x}_i \in \mathcal{X}$ and corresponding solutions $\vec{y}_i \in \mathcal{Y}$ for all $i \in \{1, \dots, d\}$, where each of the latter is determined via Eq. 20 using a respective measurement to obtain Eq. 17. Additionally, ℓ may depend on meta data ϑ such as ground-truth labels in supervised learning, depending on context. Examples for loss functions include mean squared error, cross-entropy loss, or log-likelihood.

3 Shapley values for quantum circuits

In Sect. 2.4, we have outlined a typical hybrid quantum-classical ML pipeline in form of a VQA as sketched in Fig. 1. It relies on the execution of variational quantum circuits, which are defined by a unitary operator $U(\vec{x}, \vec{\theta})$ acting on the joint Hilbert space of all q qubits that have initially been prepared in the ground state $|0\rangle$. Moreover, it depends on the k -dimensional features $\vec{x} \in \mathcal{X} \subseteq \mathbb{R}^k$ as well as the p -dimensional variational parameters $\vec{\theta} \in \mathcal{P} \subseteq \mathbb{R}^p$.

Since SVs enable model-agnostic explainability, they can be applied to QML models (i.e., models, which result from a quantum-enhanced ML pipeline) in complete analogy to the “traditional” XAI approach for classical ML, where features represent players and the model output determines the value function (Power and Guha 2024; Gil-Fuster et al. 2024).

In this manuscript, we go beyond this feature-based application and propose a more subtle use of SVs that is centered around the architecture of QML models. Specifically, presume that a hybrid ML pipeline or its resulting model contains a (possibly parameterized) circuit for quantum evaluations.

Our goal is to find an interpretation or explanation for this crucial component within the pipeline or model. For this purpose, we consider the gates (or groups of gates) as players (or their respective indices, to be more precise) and a value function that is determined by the measurement results. This approach allows us to assign a SV to every gate of the circuit with respect to the chosen value function and hence assign a corresponding contribution. Consequently, we are able to evaluate the quality of different ansätze and their building blocks in great detail. We refer to our approach as *Shapley values for quantum circuit explanations* (SVQXs)³.

The premise of SVQXs is conceptionally related to SV-based NAS in the sense that we consider building blocks of ML models as players. However, we present our approach in a way that is customly tailored for quantum computations and yet highly generic. Furthermore, we focus on its use for explainability rather than using it within a QAS framework.

In summary, we identify two approaches for applying SVs in QML as sketched in Fig. 2: either using features as players (the “traditional” XAI approach) or using gates as players (what we consider in this work with SVQXs). Both concepts are found on the same premise of a coalition game.

Game theory in the context of quantum computing has been considered before; see, e.g., Khan et al. (2018) for a recent review of (non-cooperative) quantum games. SVQXs allow to assign a contribution to individual gates as a whole, where the gates can either be parameterized with a fixed parameter value or non-parameterized. This is in contrast to other approaches that measure the importance of the gate parameters themselves, for example with gradient-based methods (Haug et al. 2021).

In the remaining part of this section, we first introduce a formal definition of SVQXs. Next, we motivate and present a selection of possible value functions that can be used for various QML use cases. We conclude with a brief summary of the key ingredients of our proposed method.

3.1 Definition of SVQXs

To implement SVQXs, we decompose the unitary operator

$$U(\vec{x}, \vec{\theta}) = \prod_{g \in \{1, \dots, G\}} U_g(\vec{x}^g, \vec{\theta}^g) := U_G(\vec{x}^G, \vec{\theta}^G) \cdots U_1(\vec{x}^1, \vec{\theta}^1) \quad (22)$$

from the quantum circuit of interest, Eq. 16, into G unitary operators $U_g(\vec{x}^g, \vec{\theta}^g)$ that represent gates (or groups of gates) within the circuit and may depend on (a possibly empty subset

³ In an earlier version of the manuscript, we used the term *quantum Shapley values* (QSVs). However, we have changed this term to better reflect the fact that we are applying classical (uncertain) Shapley values to quantum gates and quantum-based value functions without actually extending the concept of Shapley values themselves to a quantum version.

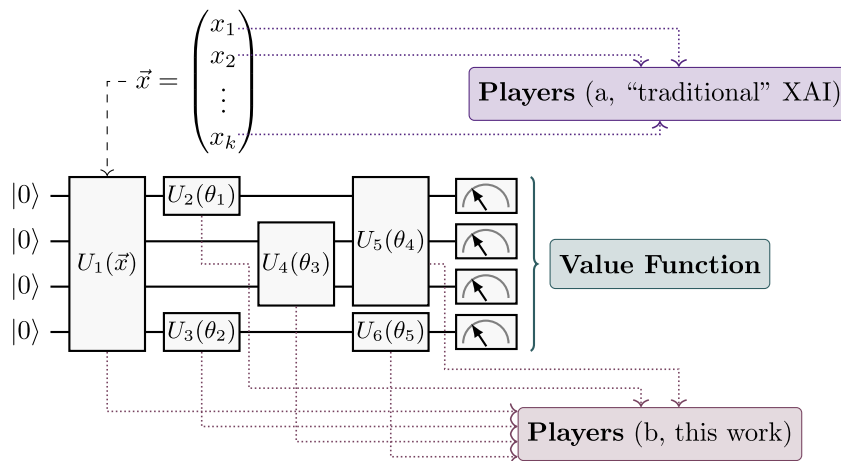


Fig. 2 Two possible approaches for SVs in QML involving variational quantum circuits: (a) SVs from classical ML for which features represent players. (b) Newly proposed SVQXs for which quantum gates represent players. In both cases, the value function, Eq. 30, is determined by the measurement results. Shown is an exemplary four-qubit variational circuit consisting of a data encoding unitary for a k -dimensional feature vector $\vec{x} := \{x_1, \dots, x_k\}$ and a set of five parameterized gates

as outlined in Fig. 1, where each parameterized gate depends on a single variational parameter $\vec{\theta}^{i+1} := \{\theta_i\} \subset \vec{\theta} \forall i \in \{1, \dots, 5\}$ with $\theta := \{\theta_1, \dots, \theta_5\}$. In terms of Fig. 22, $G = 6$, $U_1(\vec{x}^1, \vec{\theta}^1) := U_1(\vec{x})$, and $U_{1+i}(\vec{x}^{1+i}, \vec{\theta}^{1+i}) := U_{1+i}(\theta_i) \forall i \in \{1, \dots, 5\}$. For both of the SV approaches presented here, the gate parameters θ are assumed to be arbitrary but fixed

of) features $\vec{x}^g \subseteq \vec{x}$ and variational parameters $\vec{\theta}^g \subseteq \vec{\theta}$, respectively, for all $g \in \{1, \dots, G\}$. The product in Eq. 22 is to be understood in such a way that the terms are ordered as indicated.

Furthermore, we define a set

$$A := \{A_1, \dots, A_N\} \subseteq \{1, \dots, G\} \quad (23)$$

of active gates, which participate as players, whereas the remaining gates

$$R := \{1, \dots, G\} \setminus A \quad (24)$$

are treated as passive (or remaining) part of the circuit. The decomposition of the unitary operator into gates and the respective subset of active gates can be chosen at will for the use case of interest.

The corresponding coalition game involves $N = |A|$ players. Coalitions in this game represent sets of active gates that can be associated with circuits consisting of the gates of the coalition and the passive gates in R . For a given coalition S , such a circuit is defined by the unitary operator

$$U(S, R, \vec{x}, \vec{\theta}) := \prod_{g \in \{A_a \mid a \in S\} \cup R} U_g(\vec{x}^g, \vec{\theta}^g) \quad (25)$$

that yields a quantum state

$$|\Psi(S, R, \vec{x}, \vec{\theta})\rangle := U(S, R, \vec{x}, \vec{\theta})|0\rangle \quad (26)$$

in analogy to Eq. 16. The product in Eq. 25 is to be understood as ordered in the same way as in Eq. 22. Measurement results for q qubits are consequently given by

$$\vec{B}(S, R, \vec{x}, \vec{\theta}) \equiv \vec{B}(S) := \{\vec{b}_1(S), \dots, \vec{b}_s(S)\}, \quad (27)$$

in analogy to Eq. 17 with $\vec{b}_i(S) \in \{0, 1\}^q$ and

$$\vec{b}_i(S) \sim \mathbb{P}_{S, R, \vec{x}, \vec{\theta}}(\vec{b}) := \langle \Psi(S, R, \vec{x}, \vec{\theta}) | M(\vec{b}) | \Psi(S, R, \vec{x}, \vec{\theta}) \rangle \quad (28)$$

for $i \in \{1, \dots, s\}$ in analogy to Eq. 18. Hence, Eq. 19 can be written as

$$S, R, \vec{x}, \vec{\theta} \mapsto \vec{B}(S), \quad (29)$$

which also depends on the active gates (players) in S and the passive gates in R . In Fig. 3, we exemplarily outline the set of circuits that are involved in such a coalition game.

Technically, SVQXs are computed in the same way as conventional SVs via Eq. 8. Their premise is a coalitional game based on gates within a quantum circuit as players. For such a coalition game, the value function, Eq. 7, takes the form

$$v(S) = v(S; R, \vec{x}, \vec{\theta}, \vec{B}(S)). \quad (30)$$

More generally, the value function can in analogy to Eq. 14 depend on multiple feature vectors $\vec{x}_i \in \mathcal{X}$ and variational

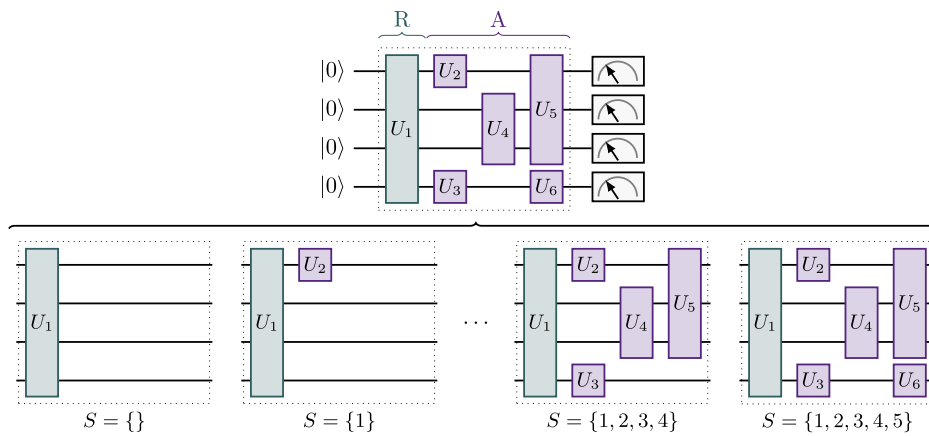


Fig. 3 Schematic representation of the set of circuits that are involved in a coalition game for SVQXs. Exemplarily, we consider the circuit from Fig. 2 as the circuit of interest with an active gate set $A = \{A_1 = 2, A_2 = 3, A_3 = 4, A_4 = 5, A_5 = 6\}$, Eq. 23, and a set of remaining gates $R = \{1\}$, Eq. 24. The original circuit of interest is shown on top, it contains the six gates $U_1, \dots, U_6$, where we omit all arguments. The index of each gate indicates the corresponding gate index

$g \in \{1, \dots, G\}$ with $G = 6$ in Eq. 22. Each coalition S can be associated with a circuit that contains the gates whose indices are included in the set $\{A_a \mid a \in S\} \cup R$ as defined in Eq. 25. Four exemplary coalitions and their corresponding circuits are shown at the bottom. The leftmost and rightmost circuits are associated with the empty and grand coalition, respectively

parameters $\vec{\theta}_i \in \mathcal{P}$. Measurement results are then obtained for each $\vec{B}(S, R, \vec{x}_i, \vec{\theta}_i)$, Eq. 27, for all $i \in \{1, \dots, d\}$, hence

$$v(S) = v(S; R, \Xi_d). \quad (31)$$

Here, we made use of the abbreviation

$$\Xi_d := \{(\vec{x}_1, \vec{\theta}_1, \vec{B}(S, R, \vec{x}_1, \vec{\theta}_1)), \dots, (\vec{x}_d, \vec{\theta}_d, \vec{B}(S, R, \vec{x}_d, \vec{\theta}_d))\}. \quad (32)$$

For $d = 1$, Eq. 31 reduces to Eq. 30. A value function may also depend on additional meta data.

In the most general case of value functions, the measurement results, Eq. 27, may refer not just to direct measurements of the circuit of interest, but instead to an alternation or modification of this circuit in the sense that the unitary from Eq. 25 is mapped onto another unitary, i.e., $U(S, R, \vec{x}, \vec{\theta}) \rightarrow U'(S, R, \vec{x}, \vec{\theta})$. This allows to realize experiments like a SWAP test (Barenco et al. 1997; Buhrman et al. 2001). The results may also depend on a particular order, for example, to realize an iterative quantum state tomography algorithm (Zambrano et al. 2020). In the next section, we discuss examples for value functions, which also include such special cases.

In addition to executing circuits on a QPU, we also consider the case where a circuit is simulated on classical hardware. Such an approach allows to perform quantum computations that are not yet feasible on NISQ devices due to hardware-related imperfections like dissipation and decoherence (Zhou et al. 2020). There are two major differences between simulating a circuit classically and running it on an actual QPU. Firstly, the simulation is noise-free (but noise can optionally be modeled Georgopoulos et al. 2021). And

secondly, the state $|\Psi(S, R, \vec{x}, \vec{\theta})\rangle$ from Eq. 26 can be determined directly without the need for a measurement. In terms of the presented formalism, a simulation corresponds to the case of infinite shots, i.e., $s \rightarrow \infty$ in Eq. 17 such that the probability distribution $\mathbb{P}_{S, R, \vec{x}, \vec{\theta}}$, Eq. 28, can be obtained with arbitrary precision for all bit strings $\vec{b} \in \{0, 1\}^q$. Alternatively, a value function for simulated circuits may also directly depend on the quantum state instead of the measurement results. An exemplary simulation-based value function is provided in the following section as well.

3.2 Value functions for SVQXs

Value functions for SVQXs, Eq. 31, can be chosen in many different ways. First of all, value functions from classical XAI can be used in the same way for QML tasks. In analogy to Eqs. 13 and 14, we can define

$$v(S) = v(S; \vec{x}, h(\vec{B}(S, R, \vec{x}, \vec{\theta}))), \quad (33)$$

for single data points $\vec{x} \in \mathbb{R}^N$ and

$$v(S) = v(S; \vec{x}_1, \dots, \vec{x}_d, h(\vec{B}(S, R, \vec{x}_1, \vec{\theta}_1)), \dots, h(\vec{B}(S, R, \vec{x}_d, \vec{\theta}_d))) \quad (34)$$

for a data set $\vec{x}_1, \dots, \vec{x}_d$ consisting of d data points $\vec{x}_i \in \mathbb{R}^N$ for $i \in \{1, \dots, d\}$. Here, we recalled h from Eq. 20. For example, a loss function of the form of Eq. 21 can be used as the value function in Eq. 34, that is,

$$v(S) = \ell(\{(\vec{x}_1, h(\vec{B}(S, R, \vec{x}_1, \vec{\theta}_1))), \dots, (\vec{x}_d, h(\vec{B}(S, R, \vec{x}_d, \vec{\theta}_d)))\}, \vartheta). \quad (35)$$

SVQXs based on such a value function allow to assign a contribution of each gate to the loss.

Moreover, more specialized value functions can be used independent of a particular ML task to evaluate certain properties of a quantum circuit. In this context, we specifically propose four typical use cases in the form of questions:

1. How uniformly is the unitary space explored by an ansatz circuit?
2. What ability has an ansatz circuit to create entangled states?
3. How does a circuit evaluation perform on NISQ hardware in comparison with an idealized classical simulator?
4. How efficiently can a circuit be executed on a specific NISQ hardware device?

In the following, we briefly outline potential value functions to answer these questions.

For the first use case, a suitable metric to quantify the uniform exploration capability of the unitary space is given by the *expressibility* (Sim et al. 2019). It corresponds to the distance between the distribution of unitaries generated by the ansatz with randomly chosen variational parameters and the maximally expressive uniform distribution of unitaries. We consider an ansatz circuit with a corresponding unitary, Eq. 16, that depends on a parameter vector $\vec{\theta}$, whereas the feature vector $\vec{x}$ is presumed to be constant. An estimator for the expressibility of this circuit $\eta(S, R, \vec{x}, c_p, c_b, \mathcal{F})$ is presented in Appendix A. Here, c_p and c_b denote parameters that determine the precision of the estimator, whereas $\mathcal{F}$ contains measurement results from SWAP tests. The corresponding value function

$$v(S) = -\eta(S, R, \vec{x}, c_p, c_b, \mathcal{F}) \leq 0 \quad (36)$$

allows to quantify the contribution of each gate to the expressibility, i.e., its exploration capability of the unitary space. This quantity is in particular independent of a specific parameter vector $\vec{\theta}$. The negative sign in Eq. 36 ensures that larger values indicate a higher expressibility, whereas smaller values indicate a lower expressibility.

To answer the question of the second use case, the *entangling capability* represents a suitable metric (Sim et al. 2019). As we present in Appendix B, it is based on the mean Meyer-Wallach entanglement measure (Meyer and Wallach 2002) for randomly drawn variational parameters. The metric can be directly used as a value function

$$v(S) = \lambda(S, R, \vec{x}, c_s, \mathcal{T}) \in [0, 1] \quad (37)$$

that allows to quantify the contribution of each gate to the overall entangling capability of the circuit, where larger val-

ues indicate more entangling capability. This quantity is in particular independent of a specific parameter vector $\vec{\theta}$. Here, c_s determines the number of drawn parameter samples, whereas $\mathcal{T}$ represents a list of measurement results for quantum state tomography.

For the third use case, we evaluate the circuit of interest for a given coalition, Eq. 25, both on an actual QPU as well as with a (noise-free) simulator running on a classical host. As briefly outlined in Appendix C, this approach allows us to use the *Hellinger fidelity* (Hellinger 1909) to quantify the degree of agreement between the measurement results from the quantum device with the results from the classical simulator based on the respective distributions of bit strings. Two effects are responsible for a potential deviation between these two results. First, the limited number of shots on the QPU and second, hardware-related uncertainty from the NISQ device that is not present on the idealized simulator. Since a lower degree of agreement consequently results from deficiencies (or inadequacies) of the QPU, the Hellinger fidelity is a measure of the severity of these deficiencies. The Hellinger fidelity can be directly used as a value function

$$v(S) = H(S, R, \vec{x}, \vec{\theta}, \vec{B}(S)) \in [0, 1] \quad (38)$$

that allows to quantify the contribution of each gate to the quantum hardware deficiencies. By definition, smaller values indicate larger deficiencies. Here, $\vec{B}(S)$ denotes the acquired measurement results, Eq. 27. In particular, the evaluation of this value function requires both a simulation of the circuit as well as its execution on a QPU.

The fourth use case considers the practical challenge of efficiently executing an arbitrary quantum circuit on a NISQ device. As explained in Appendix D, any quantum circuit must be subjected to a suitable equivalence transformation to allow execution on a specific hardware device, a process also known as *transpilation* (Anis et al. 2019). The transpilation procedure transforms a circuit into a form, where it only contains gates from the set $\mathcal{U}$ of available gates as specified by the hardware device of interest. In this sense, the transpilation can be understood as a necessary *classical* preprocessing procedure for all QPU evaluations. However, such a procedure is ambiguous by definition and can therefore lead to different gate decompositions with a varying number of gates. In general, a lower number of gates is preferable since each gate contributes to the noise of the corresponding QPU computation. As briefly outlined in Appendix D, the estimated execution efficiency $\tau(S, R, \mathcal{U}, s_1, s_2)$ represents a metric that measures how efficiently a circuit can be executed on a specific hardware device by assigning a negative penalty value to each gate based on the most favorable transpilation of the circuit. The penalty value is determined by

the penalty parameters $s_1 < 0$ and $s_2 < s_1$ for one-qubit and two-qubit gates, respectively. The corresponding value function

$$v(S) = \tau(S, R, \mathcal{U}, s_1, s_2) \leq 0 \quad (39)$$

can therefore be used to quantify the contribution of each gate to the estimated efficiency of executing the circuit on a specific hardware device, where larger values indicate a higher estimated efficiency.

In contrast to use case three, only the efficiency of the transpilation process and not the actual hardware errors that arise during execution are considered. Therefore, the premise of the final use case is different in comparison with the other three: no QPU is used to evaluate the value function, instead the transpilation is run on a CPU to determine the estimated efficiency based on the properties of a specific hardware device based on the chosen penalty function.

3.3 Summary

In conclusion, SVQXs can be defined in analogy to conventional SVs via Eq. 8 based on a coalition game in which certain (parameterized or non-parameterized) gates of a quantum circuit constitute the players, Eq. 23. The respective value functions, Eq. 30, involve the unitary operators each coalition, Eq. 25, with possible use of results from a QPU or a classical host or both. For the sake of convenience, we use the notation

$$\Phi_{(g)} := \Phi_i \text{ with } A_i = g \quad (40)$$

to refer to the SVQXs of an active gate with gate index g , Eq. 22, where $A_i \in A$, Eq. 23. Examples for value functions based on typical use cases are presented in Eqs. 35, 36, 37, 38, and 39. Value functions based on QPU calculations may suffer from intrinsic noise and are consequently described by SVs of uncertain value functions, Eq. 8, which can be evaluated, e.g., via Eqs. 10 or 11.

4 Experiments

In this section, we demonstrate the use of SVQXs to evaluate (variational) circuits. For this purpose, we perform multiple experiments on classical simulators and actual IBM quantum hardware. In total, we consider five use cases for which we recall a selection of the proposed value functions from Sect. 3.2. Three use cases are from the QML domain as a first step towards XQML: two classifiers in Sects. 4.1 and 4.2, respectively, and a generative model in Sect. 4.3. Finally, the last two use cases consider circuit transpilation as a classical preprocessing task in Sect. 4.4 and solving a combinatorial

optimization problem with a variational quantum eigensolver (VQE) in Sect. 4.5, which allows us to demonstrate the usability of SVQXs beyond XQML. The goal of this section is to present different application areas for SVQXs without going too deep into possible interpretations or even conclusive explanations of the results.

The experiments have been realized with Python using Qiskit (Anis et al. 2019) and a SVQX toolbox (Heese 2023). All data used in the experiments is publicly available online (Heese et al. 2023; Aha 1991).

We use a selection of (simulated and NISQ) QPUs to perform quantum computations. Specifically, as simulators we use (i) an idealized statevector simulator (`state_CPU`), which allows a deterministic evaluation of all amplitudes of the final state and (ii) an idealized shot simulator (`shot_CPU`), which samples a finite number of measurements (shots) from the amplitudes of the final state. Both simulators are implemented in Qiskit and run on a classical host. In contrast to the `state_CPU` simulator, the `shot_CPU` simulator has a non-deterministic behavior due to the shot noise. As NISQ devices, we use the `ibmq_ehningen` and `ibmq_oslo` QPUs. Both of these QPUs are superconducting quantum hardware devices provided by IBM Quantum through cloud services (IBM 2022a, b). Detailed specifications can be found in Appendix E.

For the estimation of SVQXs, we use either Eqs. 10 or 11. The first approach depends on the chosen number of repetitions K and the second on both K and the number of samples n . For practical purposes, we specify K and a sampling fraction $\alpha \in (0, 1]$ to fully define our estimation method. A choice of $\alpha < 1$ indicates that we use Eq. 11 with a number of samples

$$n \equiv n(\alpha) := \lceil \alpha 2^{N-1} \rceil, \quad (41)$$

whereas $\alpha = 1$ indicates that we use Eq. 10.

4.1 Quantum support vector machine

As our first use case, we consider a quantum support vector machine (QSVM) (Havlíček et al. 2019) with the task of solving a supervised classification problem on a toy data set with two-dimensional features. This QML model is based on a classical support vector machine (SVM) (Cortes et al. 1995) that operates on a kernel matrix from a QPU. That is, the elements of the kernel matrix are estimated from the transition amplitude by measuring the number of all-zero strings of a composition of two consecutive feature map circuits, each parameterized by one of the two feature vectors of interest (Havlíček et al. 2019). Consequently, both training the QSVM and making predictions with it requires the evaluation of quantum circuits. As feature map circuit, we use a two-qubit second-order Pauli-Z evolution circuit with $7r$

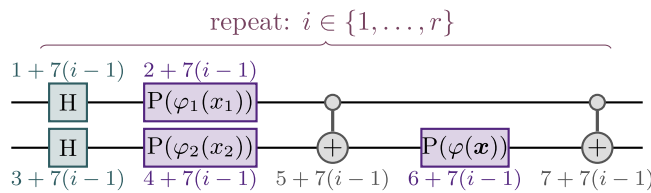


Fig. 4 Feature map circuit for the QSVM: two-qubit second-order Pauli-Z evolution circuit with r repetitions parameterized by feature vectors $\vec{x} \in \mathbb{R}^2$. We use the circuit symbols from Sect. 11 and the abbreviations from Eqs. 42 and 43. The number near each gate rep-

resents its gate index g , Eq. 22. Here and in the following, we omit initialization and measurements in circuit sketches to simplify our presentation. These operations are by default always realized as shown in Fig. 3

gates, where r denotes the number of repetitions as shown in Fig. 4. We particularly consider $r \in \{1, 2, 3\}$. For the parameterization of the circuit (with two-dimensional feature vectors $\vec{x} \in \mathbb{R}^2$), we use the abbreviations

$$\varphi_i(x_i) := 2x_i \quad (42)$$

for $i \in \{1, 2\}$ and

$$\varphi(\vec{x}) := 2(\pi - x_1)(\pi - x_2), \quad (43)$$

respectively. For a recent review of QSVMs, we refer to Gentinetta et al. (2022) and references therein.

For training and testing purposes, we use the artificial data set from Havlíček et al. (2019) with two-dimensional features $\vec{x} \in (0, 2\pi]^2 \subset \mathbb{R}^2$ and binary class labels $y \in \{0, 1\}$. By construction, this data can in principle be fully separated by the chosen feature map with $r = 2$. We choose a separation gap of 0.3, which is a hyperparameter that determines the geometrical shape of the data. In total, we uniformly draw 40 training points and 1000 test points from the feature space, both with an equal distribution of zero and one class labels, as shown in Fig. 5.

For this use case, all gates are active gates such that $A = \mathcal{G}$ and $R = \emptyset$ as defined in Eqs. 23 and 24. Furthermore, $\Phi_{(g)} =$

Φ_i according to Eq. 40. The considered value function $v(S)$ is defined by the following procedure:

1. Evaluate the kernel matrix for the training data set based on the feature map circuit, Fig. 4, that contains only gates from S , Eq. 25.
2. Train the QSVM using this kernel matrix. The training algorithm is deterministic.
3. Evaluate the accuracy of the trained model on the test data set to determine $v(S) = \text{acc}$ using the same feature map circuit as before.

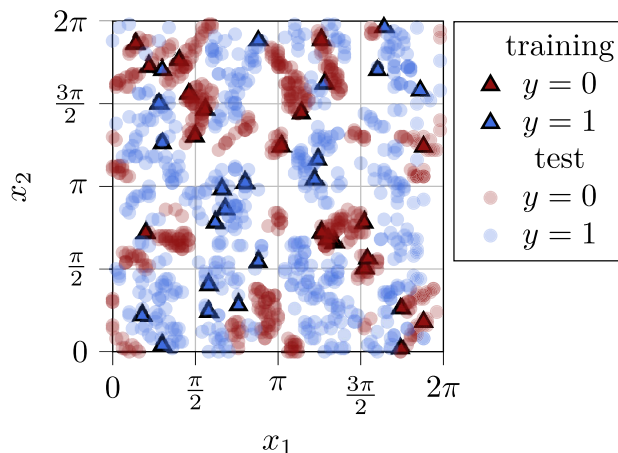
The accuracy is defined in the usual sense as

$$\text{acc} := \frac{\text{number of correctly predicted test labels}}{\text{total number of test points}} \in [0, 1] \quad (44)$$

and is therefore of the form of Eq. 35. This value function allows to quantify the contribution of each gate to the kernel matrix for a successful training of the QSVM, where larger values indicate a better model performance.

We run all evaluations on the `state_CPU` simulator. Since the training algorithm of the QSVM is also deterministic, no randomness is involved in the evaluation of the value function with the consequence that $\hat{\Phi}_i^1 = \Phi_i = \phi_i$ according to Eqs. 2, 8, and 10. For $r \in \{1, 2\}$, we therefore use

Fig. 5 Artificial test and training data sets for the supervised classification problem with two-dimensional features $\vec{x} \in (0, 2\pi]^2$ and binary labels $y \in \{0, 1\}$. We use a QSVM to solve this problem



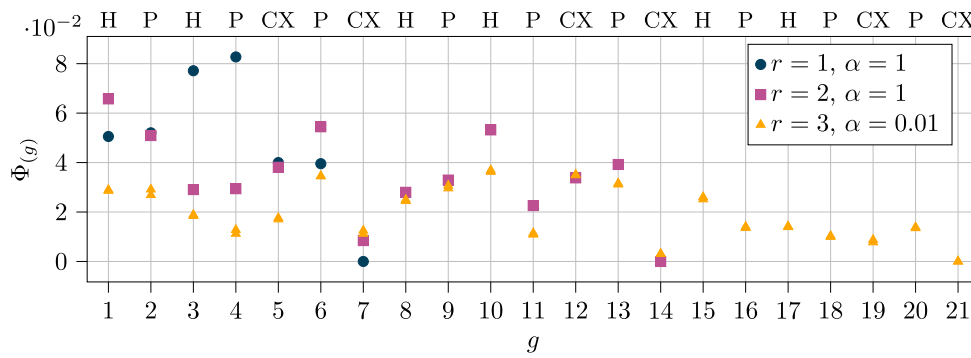


Fig. 6 SVQXs $\Phi_{(g)}$, Eq. 40, for the feature map circuit of a QSVM, Fig. 4, with a value function corresponding to the retrained test accuracy of the corresponding model. A larger SVQX is consequently more favorable. We consider three different numbers of repetitions $r \in \{1, 2, 3\}$. The gate index g , Eq. 22, is shown on the bottom, whereas the corresponding gate name is shown on the top of the plot. All evaluations are run on the `state_CPU` simulator, which leads to deterministic value

functions. We therefore use $K = 1$ repetitions for their evaluation. For $r \in \{1, 2\}$, we decide not to sample value functions by choosing a sample fraction $\alpha = 1$, Eq. 41, which leads to deterministic SVQXs via Eq. 10. For $r = 3$, we choose a value function sample with $\alpha = 0.01$ and perform two independent runs, for each of which we draw different samples using Eq. 11. We plot the results of both runs with the same symbols

$K = 1$ and $\alpha = 1$, Eq. 41. On the other hand, we choose $K = 1$ and $\alpha = 0.01$ for $r = 3$ to reduce the computational effort, Eq. 11, and perform two independent runs, for each

of which we draw different samples. The resulting SVQXs for the feature map circuit, Fig. 4, with different numbers of repetitions r are presented in Fig. 6. The resulting test accu-

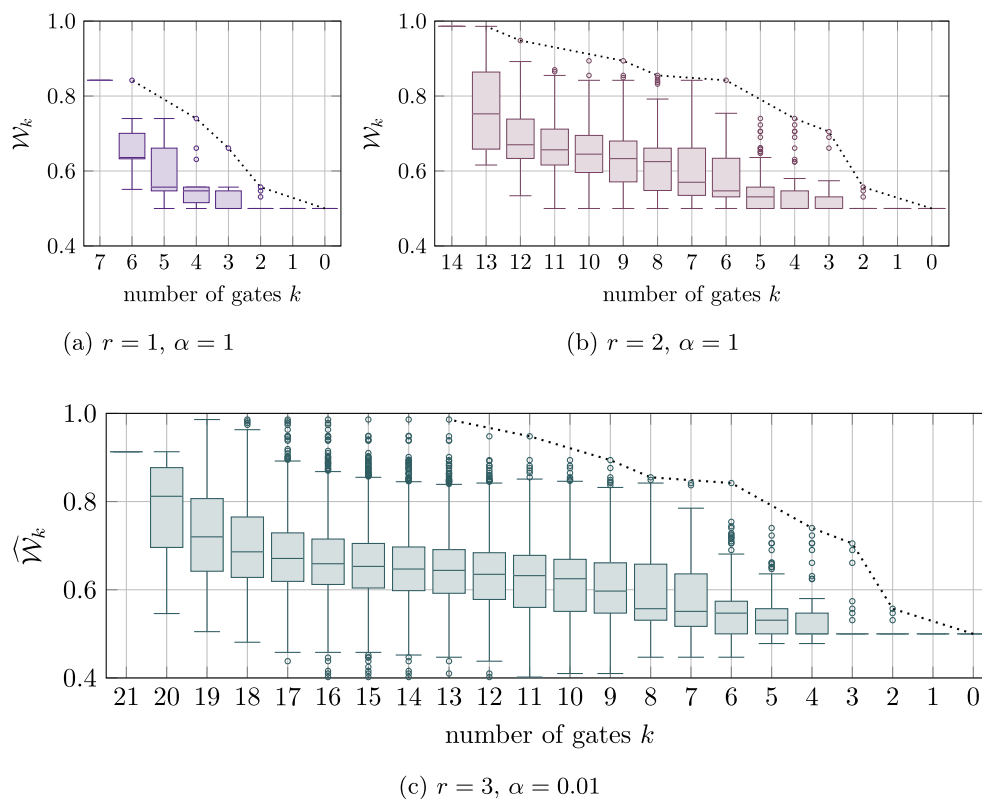


Fig. 7 Distribution of value functions $\mathcal{W}_k$, Eq. 45, for different number of gates k in form of box plots. The results are obtained for the experiments from Fig. 6. We show three different numbers of repetitions $r \in \{1, 2, 3\}$. All evaluations are run on a `state_CPU` simulator with $K = 1$. The dotted line indicates the Pareto frontier for a multi-

criteria circuit pruning problem with the goal to minimize the number of gates and to maximize the test accuracy of the QSVM. For $r = 3$, we consider the distribution of sampled value functions $\hat{\mathcal{W}}_k$ for one run. The corresponding Pareto frontier is an approximation since not all value functions have been evaluated

racies read $\text{acc} \approx 0.842$ for $r = 1$, $\text{acc} \approx 0.986$ for $r = 2$, and $\text{acc} \approx 0.913$ for $r = 3$.

We find that the resulting SVQXs exhibit a rich structure. First of all, we can observe that the SVQXs for the last CX gate of a feature map circuit vanish since it has no effect on the measured number of all-zero strings and therefore on the value function. Intermediate CX gates, on the other hand, have non-vanishing SVQXs. Overall, the CX gates at the end of a repetition (i.e., $g \in \{7, 14, 21\}$) have small SVQXs. Supplementary, we show the corresponding distributions of marginal contributions for the evaluation of the SVQXs from Fig. 6 in Appendix G.

Another perspective on SVQXs is to analyze the corresponding value function distributions for subsets of gates. For this purpose, we consider the multiset of value functions

$$\mathcal{W}_k := \{v \mid v = v(S) \in \mathbb{R} \forall S \subseteq \mathcal{G} \wedge |S| = k\} \quad (45)$$

given k (active) gates. That is, every element in $\mathcal{W}_k$ corresponds to a retrained test accuracy for a QSVM with a specific feature map circuit (consisting of k gates in total). To determine $\mathcal{W}_k$, all value functions have to be evaluated, which corresponds to $\alpha = 1$. In case of $\alpha < 1$, we consider the multiset of sampled value functions $\hat{\mathcal{W}}_k$ instead. We plot the distribution of $\mathcal{W}_k$ and $\hat{\mathcal{W}}_k$, respectively, for the results from Fig. 6 as box plots in Fig. 7.

The plots confirm that one gate (the last CX) can be removed without lowering the test accuracy. Furthermore, for $r = 3$ removing up to eight gates can even lead to an

improvement of the score. The trade-off between the number of gates and the test accuracy can be considered as a multi-criteria optimization problem for circuit pruning (Sim et al. 2021). The corresponding Pareto frontier is indicated as a dotted line in Fig. 7. This analysis can be understood as a very simple form of QAS that allows to select the optimal gate configuration for a certain task.

So far, SVQXs have allowed us to gain certain insights into the influence of the gates of a feature map circuit on the test accuracy of a corresponding QSVM. In order to investigate the significance of these insights and the general robustness of SVQXs, we conduct three additional tests in the following.

First, we determine the influence of the sampling fraction α , Eq. 41, that determines the number of sampled value functions. For this purpose, we perform three evaluations of the SVQXs from Fig. 6 using different values of α and $K = 1$ for each of the repetitions $r \in \{2, 3\}$. Since the evaluations are performed on a `state_CPU` simulator, the only source of randomness originates from the sampling of value functions. The results are presented in Fig. 8.

We find that the mean values of the SVQXs converge for increased values of α , whereas their standard deviation decreases, as expected. The standard deviation is in particular different for each gate.

As a second experiment, we test the generality and robustness of the result by evaluating the SVQXs from Fig. 6 for different data sets. Specifically, we consider the previously used training and test data sets and, in addition, four other training and test data sets of the same size that have been

Fig. 8 Influence of the number of sampled value functions that is determined by the sampling fraction α , Eq. 41, on the SVQXs from Fig. 6 for two repetitions $r \in \{2, 3\}$. For $r = 2$ and $\alpha = 1$, no randomness is involved so that we can show the exact results. For $r = 3$ and $\alpha = 1 \times 10^{-2}$, we plot both results of two independent runs for each case with the same symbols. In all other cases, we show the mean and one standard deviation over three independent runs. The evaluations are performed on a `state_CPU` simulator with $K = 1$

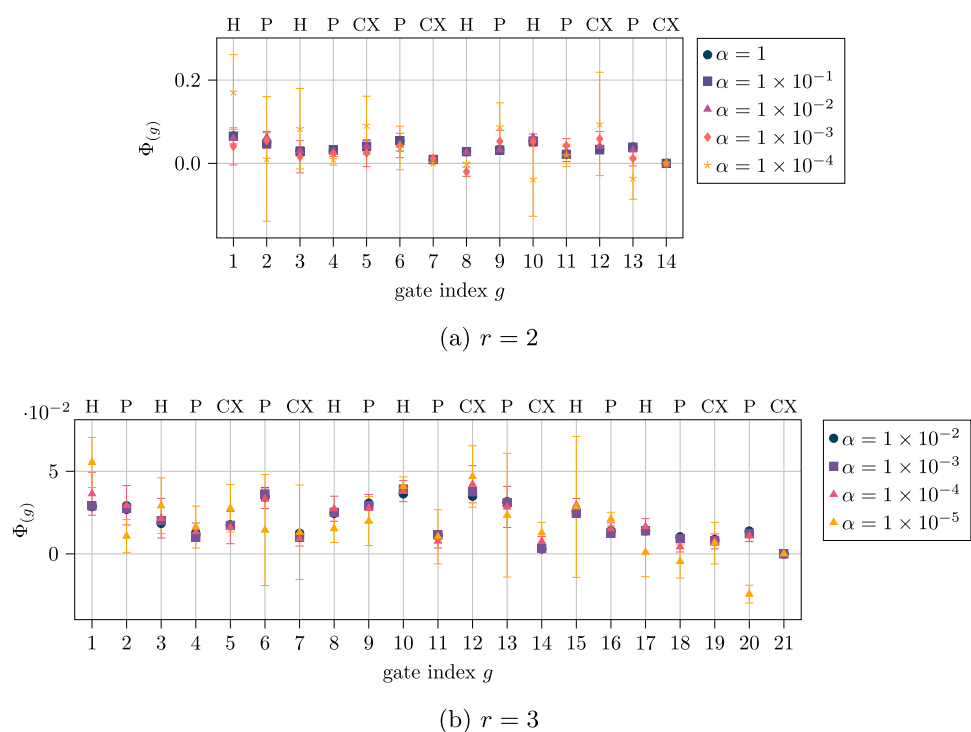
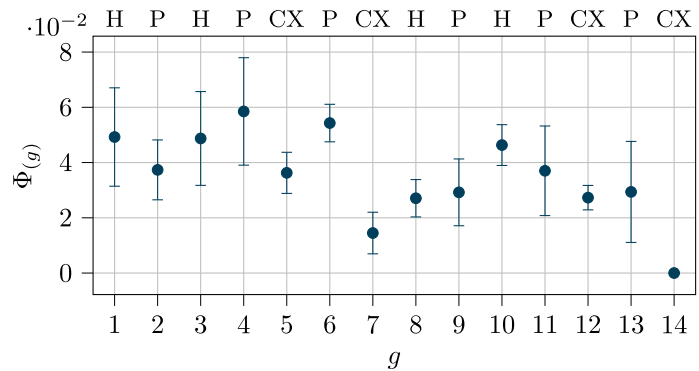


Fig. 9 Influence of the data sampling on the resulting SVQXs. We plot SVQXs for the QSVM with $r = 2$ in analogy to Fig. 6 based on five different test and training data sets (obtained from sampling the data distribution with different random seeds). We show the mean and one standard deviation. The evaluations are performed on a `state_CPU` simulator with $K = 1$ and $\alpha = 1$



sampled from the same data distribution with different random seeds. We perform the evaluations on a `state_CPU` simulator with $K = 1$ and $\alpha = 1$, which eliminates all randomness. The results are presented in Fig. 9.

From these results, it becomes apparent that the SVQXs for some gates are more susceptible to a change of the data than others. In particular, for $g = 4$, $g = 11$, and $g = 13$ we find a large standard deviation. This is no surprise since all of these gates are P gates that are directly influenced by the choice of data. Moreover, the initial H gates with $g = 1$ and $g = 3$ show a similarly large standard deviation, indicating that their contribution is also data-dependent.

Finally, in a third experiment, we investigate the influence of noise from a NISQ QPU. To this end, we choose $r = 1$ and perform six evaluations on the `ibmq_ehningen` QPU with $K = 1$ and $\alpha = 1$. Therefore, the only source of randomness is errors from the QPU. These errors (unavoidably) lead to uncertain value functions in the sense of Eq. 7. For comparison, we also perform an evaluation on the `state_CPU` simulator. The results are presented in Fig. 10.

Clearly, the results show that the influence of the errors from the NISQ QPU on the SVQXs is not significant.

4.2 Quantum neural network

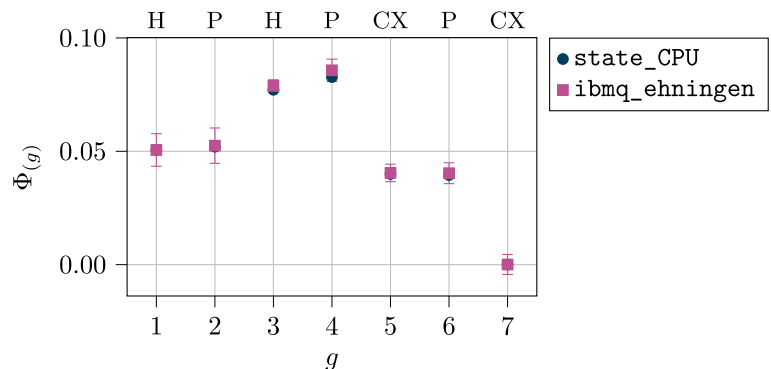
As our second use case, we consider a quantum neural network (QNN) with the task to perform a binary classification

of a toy data set with two-dimensional features. The data set contains 20 data points and is visualized in Fig. 11. By construction, it is linearly separable in the feature space. For reasons of simplicity, we use the same data set for test and training purposes. The chosen quantum circuit of the QNN is shown in Fig. 12. It is parameterized both by feature vectors $\vec{x} \in \mathbb{R}^2$ and trainable parameters $\vec{\theta} \in \mathbb{R}^4$. The prediction of the class label is determined by the binary measurement result of the first qubit.

We train the QNN once on the proposed data set with a COBYLA optimizer (Powell 2007) to determine a fixed value for the parameters $\vec{\theta} = \vec{\theta}_{\text{trained}} \approx (3.860, -1.070, -1.583, 0.860)$. The resulting accuracy of the classifier, Eq. 44, is 80%. For the evaluation of SVQXs, we choose the active gate set $A = \{2, 4, 5, \dots, 19\}$, Eq. 23, and a set of remaining gates $R = \{1, 3\}$, Eq. 24. Hence, all gates except for the initial H gates are treated as active. The considered value function $v(S)$ is defined by the one-shot test accuracy of the QNN circuit that contains only gates from $S \cup R$, Eq. 25. That is, we perform one shot for every data point to predict the class label (by parameterizing the QNN circuit with the current feature vector as $\vec{x}$ and the trained parameters $\vec{\theta}_{\text{trained}}$) and evaluate the accuracy via Eq. 44. Using the one-shot test accuracy allows us to investigate the shot noise further below.

We run all evaluations on the `shot_CPU` simulator. Therefore, the only source of randomness is the (simulated) shot noise. The shot noise leads to uncertain value func-

Fig. 10 Influence of uncertainty from NISQ hardware on the resulting SVQXs. We plot SVQXs for the QSVM with $r = 1$ in analogy to Fig. 6. All results are evaluated either on a `state_CPU` simulator or the `ibmq_ehningen` QPU with $K = 1$ and $\alpha = 1$. For `ibmq_ehningen`, we show the mean value and five standard deviations over six independent runs



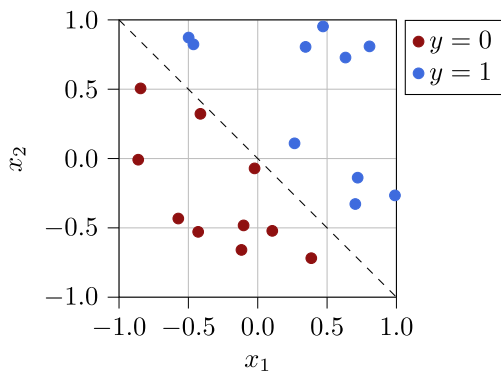


Fig. 11 Toy data for the supervised classification problem consisting of 20 data points with two-dimensional features $\vec{x} \in [-1, 1]^2$ and binary labels $y \in \{0, 1\}$. We use a QNN to solve this problem

tions in the sense of Eq. 7. The resulting SVQXs obtained for $K = 32$ and $\alpha = 0.01$, Eq. 41, are presented in Fig. 13.

We find that SVQXs show significant differences between certain gates. In particular, the gates with $g \in \{7, 8\}$ attain particularly large SVQXs, whereas the gates with $g \in \{6, 11, 14, 19\}$ attain particularly low values. Specifically, for $g = 19$ this is no surprise since the prediction of the QNN is based on the measurement of the first qubit such that the last rotation gate on the second qubit has no contribution to the prediction, i.e., the accuracy of the model.

Our choice of value function and QPU allows us to investigate the shot noise of this experiment. For this purpose, we perform several additional runs with $K \in \{1, 8, 16, 32\}$ repetitions. The resulting SVQXs are shown in Fig. 14.

For $K = 1$ the value function is based on a single one-shot prediction for each data point, which leads to a large uncertainty. As expected, the shot noise becomes less significant for a higher number of repetitions K . The plot indicates that between $K = 16$ and $K = 32$, no major improvement of the results can be observed, which implies that $K = 16$ repetitions are already sufficient to capture the uncertainty of the shot noise in this experiment.

4.3 Quantum generative adversarial network

As third use case, we consider a quantum generative adversarial network (QGAN) with the task of generating a discretized

log-normal distribution as described in Zoufal et al. (2019). For a recent review of QGANs we refer to Kiani et al. (2022). We use the circuit shown in Fig. 15 as our generative model. It consists of three qubits and can therefore produce $2^3 = 8$ different amplitudes.

First of all, we train the QGAN by suitably optimizing its trainable parameters. As a result, we find $\vec{\theta} = \vec{\theta}_{\text{trained}} \approx (-1.328, -0.155, 3.025, 1.424, 0.427, 1.620, 0.980, 1.495, 1.461)$. The QGAN output in comparison with the ground truth is shown in Fig. 16. In the following, we only consider the trained QGAN.

As active gates for the evaluation of SVQXs, we choose all gates except for the initial three H gates $R = \{1, 3, 7\}$, Eq. 24, i.e., $A = \{2, 4, 5, 6, 8, 9, \dots, 16\}$, Eq. 23. For the value function $v(S)$, we choose the Hellinger fidelity, Eq. 38, that results from a comparison of the QGAN output from the `state_CPU` simulator versus the QGAN output when evaluated on the NISQ QPU `ibm_oslo`. The value function is consequently uncertain in the sense of Eq. 7 and allows us to assess hardware errors. For `ibm_oslo`, we choose four different configurations. Specifically, we map the logical qubits to the physical qubits of the backend in two different ways. First, the three logical qubits of the QGAN circuit (from top to bottom in Fig. 15) are mapped to the physical qubits 0-1-2 of `ibm_oslo`.

Second, the physical qubits 3-5-4 are chosen. The two sets of physical qubits are also shown in Fig. 18a. For each set, we perform an evaluation with and without measurement error mitigation (MEM). The MEM method we use here requires a preprocessing step (independent of the QGAN circuit) in which the fraction of bit flips for each combination of measured ones and zeros is determined such that a linear transformation to any subsequent measurement result realizes the MEM (Anis et al. 2019; Nation et al. 2021). We perform three independent runs with $K = 3$ and $\alpha = 1$, Eq. 41. The results are shown in Fig. 17.

Comparing the two instances with and without MEM, we find that the chosen mitigation approach does not consistently increase the SVQXs but may in some cases also lead to a decrease. To further analyze our results, we require a frame of reference. For this purpose, we can use the error information about the hardware device that is provided by IBM (IBM 2022a). For technical reasons, these errors may change over

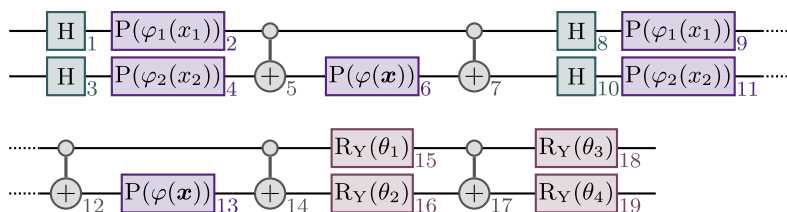


Fig. 12 Circuit for the QNN parameterized by trainable parameters $\vec{\theta} \in \mathbb{R}^4$ and feature vectors $\vec{x} \in \mathbb{R}^2$. We use the circuit symbols from Appendix F and the abbreviations from Eqs. 42 and 43. The number near each gate represents its gate index g , Eq. 22

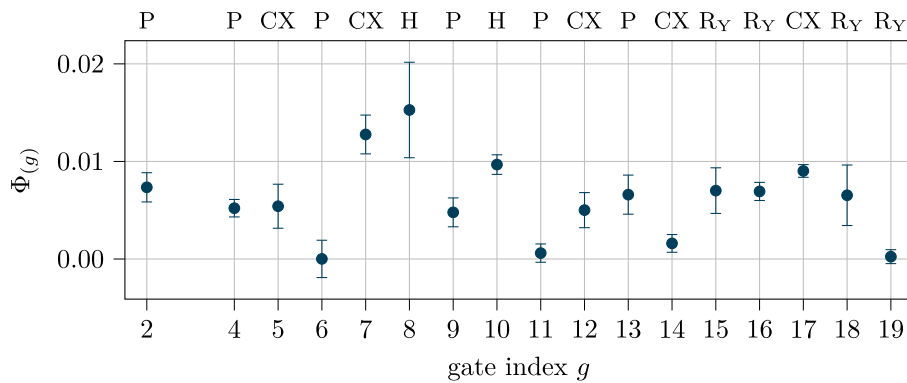


Fig. 13 SVQXs $\Phi_{(g)}$, Eq. 40, for the feature map circuit of a QNN, Fig. 12, with a value function corresponding to the one-shot accuracy of the QNN without retraining. A larger SVQX is consequently more favorable. In analogy to Fig. 6, the gate index g , Eq. 22, is shown on the bottom, whereas the corresponding gate name is shown on the top of

the plot. For the gates with $g \in R = \{1, 3\}$, no SVQX is determined. All evaluations are run on the `shot_CPU` simulator, which includes (simulated) shot noise. We use $K = 32$, $\alpha = 0.01$, Eq. 41, and show mean and one standard deviation over five independent runs

time and are therefore updated regularly (Baheri et al. 2022), which is why we collect a set of errors over the duration of the experiments. Specifically, for each of the three runs from the four configurations shown in Fig. 17, we fetch the error information every hour during the total runtime (including waiting times and the execution of MEM) as summarized in Appendix H. The results are shown in Fig. 18b and c.

Clearly, the physical qubits 0, 1, and 2 exhibit a smaller mean (and standard deviation) of the single gate errors than 3, 4, and 5. Similarly, the CX errors of the physical qubit pairs 0-1, 1-2, and 3-5, show a considerably smaller mean (and standard deviation) than 5-4. From these results, one would expect that the configuration 0-1-2 should lead to significantly better results than the configuration 3-5-4.

If we compare these observations to our results from Fig. 17, it becomes apparent that the ordering of the errors provided by IBM is not always reflected by the ordering of the corresponding SVQXs. This can be seen for example when comparing $g = 2$ with $g = 6$ and $g = 9$ with $g = 14$,

respectively. This result might be a fact of the fundamental difference between the two error definitions: Our proposed error measure via SVQXs is context-aware in the sense that it is defined for specific gates within a specific circuit, whereas the errors provided by IBM can in contrast be considered as generic. A more detailed comparison is a possible starting point for further studies, which may also take into account recent error characterizations based on discrete time crystals that suggest that “benchmarks computed infrequently are not indicative of the real-world performance of a quantum processor” (Zhang and Nation 2023).

4.4 Transpilation

The fourth use case addresses circuit transpilation as a form of classical preprocessing. Specifically, we consider the transpilation of two types of circuits:

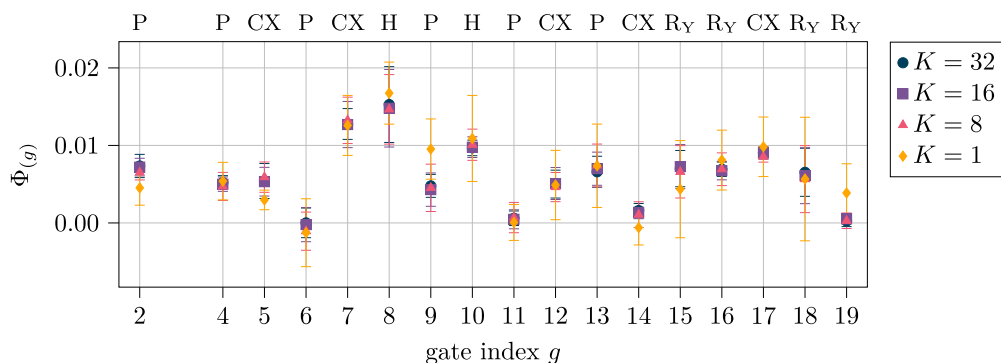


Fig. 14 SVQXs $\Phi_{(g)}$, Eq. 40, for the feature map circuit of a QNN in analogy to Fig. 13 with the goal to compare different numbers of repetitions $K \in \{1, 8, 16, 32\}$ to investigate the influence of shot noise. We

choose a sample fraction of $\alpha = 0.01$, Eq. 41. All evaluations are run on the `shot_CPU` simulator. Shown are mean and one standard deviation over five independent runs

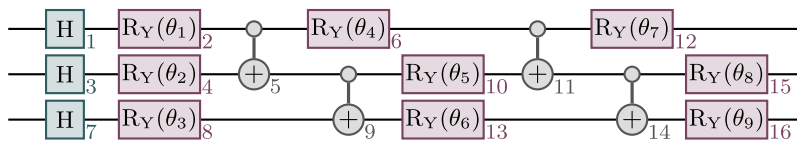


Fig. 15 Circuit for the QGAN with trainable parameters $\vec{\theta} \in \mathbb{R}^9$. We use the circuit symbols from Appendix F and the abbreviations from Eq. 42 and 43. The number near each gate represents its gate index g , Eq. 22

- A quantum decision tree classifier (QTree) with four decision branches trained on the binarized tic-tac-toe endgame data set (Aha 1991) as described in Heese et al. (2022). The model requires 16 qubits. For the implementation, we use (Heese 2022).
- Quantum Fourier Transform (QFTs) (Coppersmith 2002) for three to five qubits.

The respective circuits are shown in Fig. 19, where we also list the choice of active gates A , Eq. 23, and remaining gates R , Eq. 24, respectively. A QTree consists of an initialization layer and subsequent decision layers that represent the binary decisions within the tree model from the root to the branches.

As value function $v(S)$, we choose the transpilation-based estimated execution efficiency, Eq. 39, with penalty parameters $s_1 = -1$ and $s_2 = -10$. Its evaluation is performed exclusively on a classical host, where we repeat the transpilation process 50 times and take the best result. No measurements of the circuits take place for the evaluation. Transpilation is performed with respect to the `ibmq_ehningen` QPU using the Qiskit transpiler. Randomness is introduced by the heuristic transpilation algorithm that leads to an uncertain value function in the sense of Eq. 7. This is mitigated, to some extent, by our proposed approach of selecting the best transpilation result out of multiple repetitions. We choose $K = 1$ as well as $\alpha = 1$ for QTree and $\alpha = 0.01$ for QFT, Eq. 41. In total, we perform three independent runs for each QFT and the QTree circuit, respectively, with random sampling of value functions and random transpiler seeds. The resulting SVQXs are shown in Fig. 20.

The QTree results indicate, as expected, that deeper decision tree layers have smaller SVQXs since they involve an exponentially growing number of branches and therefore become increasingly more complex. Specifically, the plot

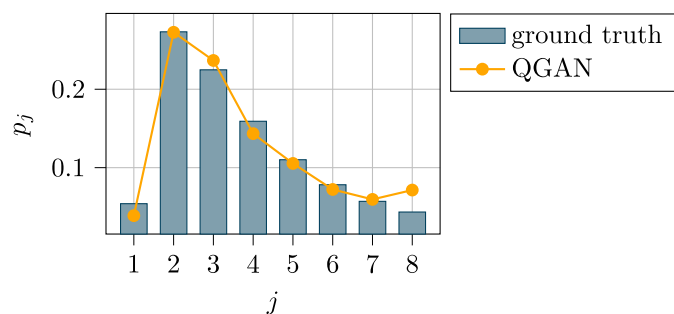
suggests an approximately exponential decrease of SVQXs for deeper layers. For the QFT, we find that specific H and CP gates for $g \in \{1, 3, 6, 10, 15\}$ exhibit larger SVQXs. On the other hand, all remaining gates (which particularly includes all S gates) show smaller SVQXs. Interestingly, for $n = 3$ the SVQXs of the gates with $g \in \{4, 5, 7\}$ have a comparably large standard deviation, whereas the standard deviation of almost all other gates is negligible.

4.5 Variational quantum eigensolver

In this final use case, we empirically investigate the usage of SVQXs for a VQE to solve an optimization task that is not from the domain of QML. Specifically, we consider a max-cut problem for the exemplary graph shown in Fig. 21. This binary optimization problem has the goal of finding a bipartite subgraph of the original graph with as many edges as possible. It is NP-hard by definition and therefore no polynomial-time algorithms are known. We use the Quantum Approximate Optimization Algorithm (QAOA) to solve this problem on a QPU (Farhi et al. 2014; Moll et al. 2018; Hadfield et al. 2019). The respective circuit is shown in Fig. 22 and depends on the chosen depth r that defines the number of repetitions for cost layers (or cost unitaries) and mixing layers (or mixing unitaries). By design, each layer consists of multiple gates, but we do not consider the internal structure for this experiment.

As active gates for the evaluation of SVQXs, we choose all problem and mixing layers. Thus, the active gate set is formally given by $A = \{8 + 2(i - 1) + k \mid i \in r \wedge k \in \{0, 1\}\}$, Eq. 23, whereas the set of remaining gates reads $R = \{1, \dots, 7\}$, Eq. 24. We consider Quantum Approximate Optimization Algorithm QAOA circuits of different depth $r \in \{1, \dots, 7\}$. To evaluate SVQXs, we first solve the opti-

Fig. 16 Output of the trained QGAN and corresponding ground truth for the discretized log-normal distribution p_j with a resolution of eight values j . The QGAN is evaluated on a `state_CPU` simulator



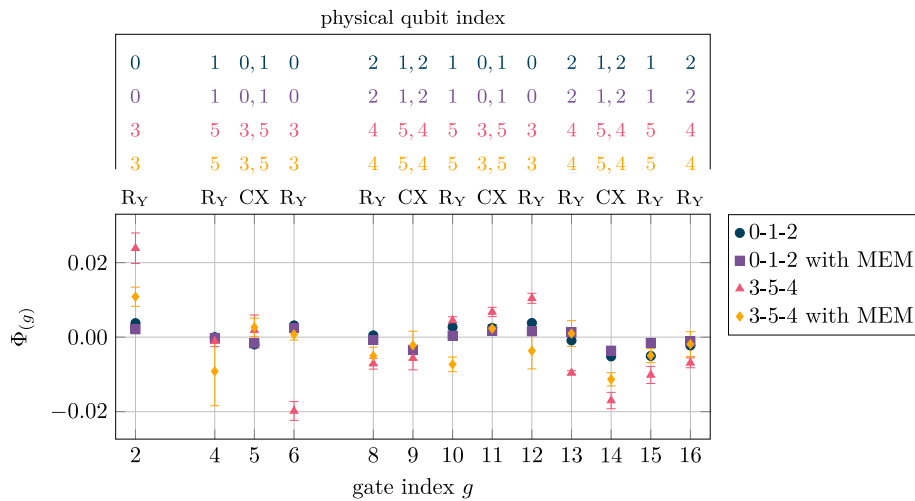
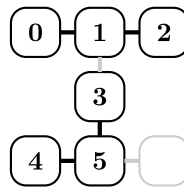


Fig. 17 SVQXs $\Phi_{(g)}$, Eq. 40, for the QGAN circuit from Fig. 15 with respect to a value function that corresponds to the Hellinger fidelity, Eq. 38, of the QGAN output on the `state_CPU` simulator versus the QGAN output on `ibm_oslo` for different configurations. First, using the physical qubits 0-1-2 and second, using the physical qubits 3-5-4, each with and without MEM. We show the mean and one standard deviation over three independent runs for each configuration. A larger SVQX

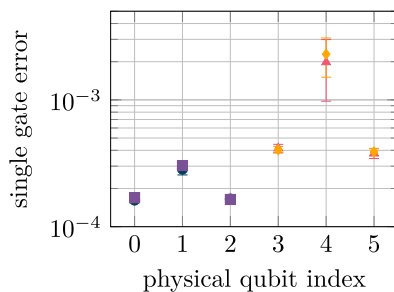
is more favorable. For all instances, we choose $K = 3$ and $\alpha = 1$. In analogy to the previous experiments, the gate index g , Eq. 22, is shown on the bottom of the plot. In addition, we also list the physical qubit indices over each gate to indicate on which physical qubits it acts. For the gates with $g \in R = \{1, 3, 7\}$, no SVQX is determined

mization problem with a COBYLA optimizer for each depth to obtain parameters $\vec{\theta} = \vec{\theta}_{\text{optimal}}$. To investigate the generality of the results, we perform three independent optimization

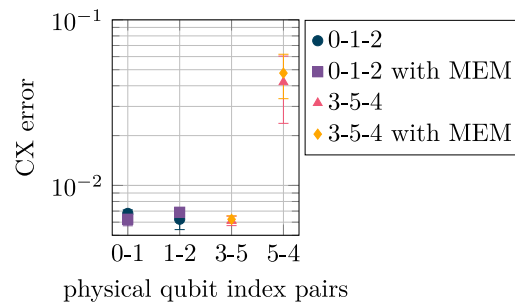
runs in total for each depth with different random seeds. That is, we obtain three different sets of optimal parameters for each depth. The resulting optimization goals are



(a) Connectivity map of `ibm_oslo`, also see Section E.



(b) Single gate errors for `ibm_oslo` as provided by IBM.



(c) CX errors for `ibm_oslo` as provided by IBM.

Fig. 18 Gate errors as provided by IBM for `ibm_oslo` (plotted with logarithmic scale). In **a**, we show the connectivity map of `ibm_oslo` with physical qubit indices and connections between qubits that allow the realization of joint CX gates, where we highlight the two sets of physical qubits (0-1-2 and 3-5-4) that are used for the results in Fig. 17.

For each run from Fig. 17, we collect the respective errors that are provided by IBM (IBM 2022a). Specifically, we fetch the error information for the hardware device every hour during the total runtime; also see Appendix H. The results are presented in **b** and **c**, respectively, where we show the mean and one standard deviation over all error samples

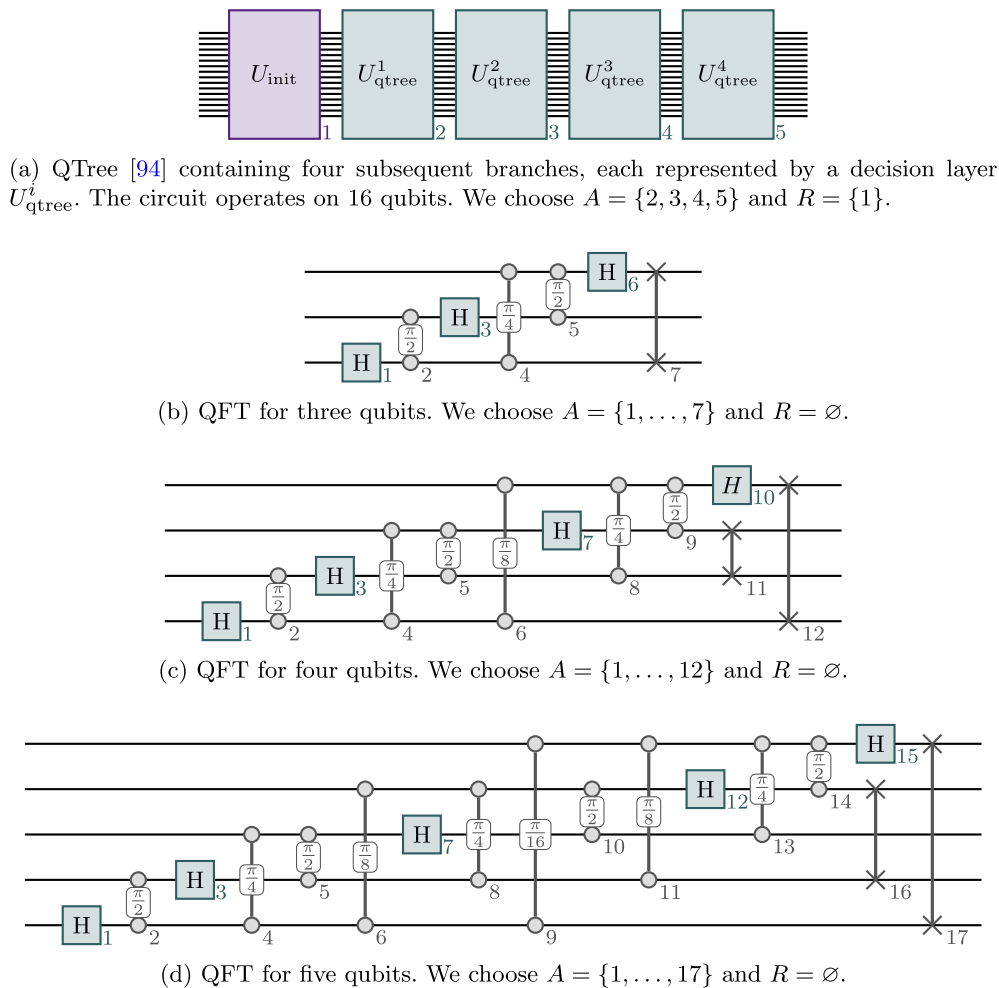


Fig. 19 Circuits for the purpose of evaluating the estimated execution efficiency. We specifically consider a QTree and three QFTs for a different number of qubits. We use the circuit symbols from Appendix F. The QTree circuit contains an initialization layer (U_{init}) and decision

layers (U_{qtree}^i for $i \in \{1, \dots, 4\}$). The number near each gate represents its gate index g , Eq. 22. For each circuit, we list the choice of active gates A , Eq. 23, and remaining gates R , Eq. 24

listed in Appendix I. Based on these fixed parameters, we evaluate the SVQXs for the respective circuits. In total, we consider three different value functions $v(S)$ for this purpose: (i) expressibility, Eq. 36, (ii) entangling capability, Eq. 37, and (iii) energy corresponding to the expectation value of the cost Hamiltonian

$$v(S) = \langle \Psi(S, R, \vec{\theta}) | \mathcal{H} | \Psi(S, R, \vec{\theta}) \rangle, \quad (46)$$

where we recall Eq. 26. The problem Hamiltonian $\mathcal{H}$ is defined in the usual way such that its ground state encodes the solution to the proposed max-cut problem.

We run all evaluations on the `state_CPU` simulator and therefore no randomness is involved in the evaluation of the value functions. We choose $K = 1$ and $\alpha = 1$ for $r \in \{1, 2, 3\}$, $\alpha = 0.1$ for $r \in \{4, 5\}$, $\alpha = 0.01$ for $r = 6$, and $\alpha = 0.001$ for $r = 7$, Eq. 41. For each repetition number, we conduct three independent runs with different random seeds

for the algorithm optimizer as well as a different sample in the sense of Eq. 11. The resulting SVQXs are presented in Fig. 23.

The SVQXs for the expressibility and entangling capability both reveal a similar structure, which clearly shows that cost layers have a dominating contribution, whereas mixing layers have a vanishing contribution. For the energy, no clear structure of the respective SVQXs can be identified and, in addition, the standard deviations over different independent optimization runs are also relatively large. Hence, we can conclude that all gates attribute similarly to the expectation value independent of the depth and their specific contribution might vary depending on the optimized parameters. This in particular means that the mixing layers contribute similarly to cost layers and therefore, pruning mixing layers from the circuit is not expected to bring a general benefit of the algorithm as can be expected from the theory. Summarized, we find from our observations that the exploration of the uni-

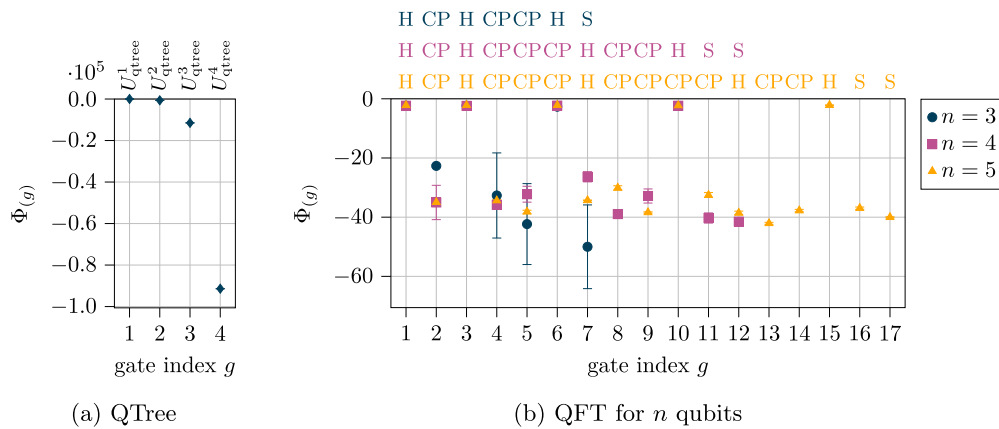


Fig. 20 SVQXs $\Phi_{(g)}$, Eq. 40, for the QTree and QFT circuits, respectively, from Fig. 19 with a value function that corresponds to the transpilation-based estimated execution efficiency, Eq. 39, with respect to the `ibmq_ehningen` QPU. A larger SVQX is more favorable. In analogy to the previous experiments, the gate index g , Eq. 22, is shown

on the bottom, whereas the corresponding gate (or layer) name is shown on the top of the plot. All evaluations are run on a classical host with $K = 1$. We choose $\alpha = 1$ for QTree and $\alpha = 0.01$ for QFT, respectively. In total, we perform three independent runs and show the mean value and one standard deviation (not visible in (a))

tary space with entangled qubits—i.e., the search space—is entirely driven by the cost layers (as the informative part of the algorithm containing the problem structure), whereas the mixing layers (as the uninformative part containing a generic structure) help to minimize the expectation value without contributing to an extension of the search space. Based on these results one could also argue that expressibility and entangling capability are both not necessarily required for energy minimization. And indeed, high expressibility can lead to flatter cost landscapes that hamper optimization (Holmes et al. 2022).

To further investigate the importance of the circuit depth for the entangling capability and expressibility, we conduct a brief analysis of the sampled value function distribution depending on the number of active gates (i.e., layers) accord-

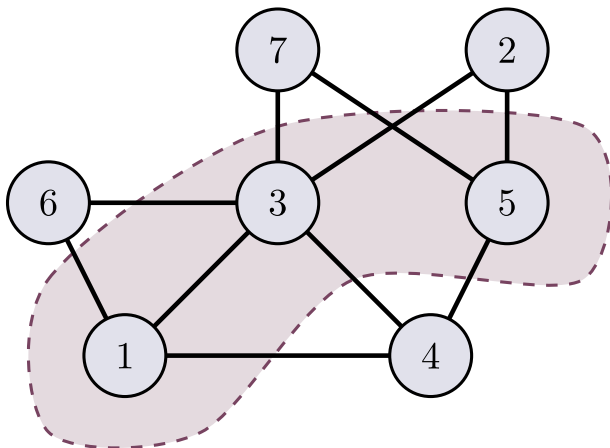


Fig. 21 Exemplary graph for the max-cut problem consisting of seven vertices (each labeled by its unique index) and ten edges. The dashed line represents the optimal cut that leads to the unique solution to the problem in form of two subgraphs that are connected by nine edges

ing to Eq. 45 in analogy to Fig. 7. For this purpose, we limit ourselves to the case $r = 7$. In this analysis, we do not consider the energy function since (in contrast to the entangling capability and the expressibility) it depends on the optimization parameters which have been determined beforehand based on the complete circuit such that starting with a pruned circuit might converge to a different set of optimization parameters with a different energy. The results are shown in Fig. 24.

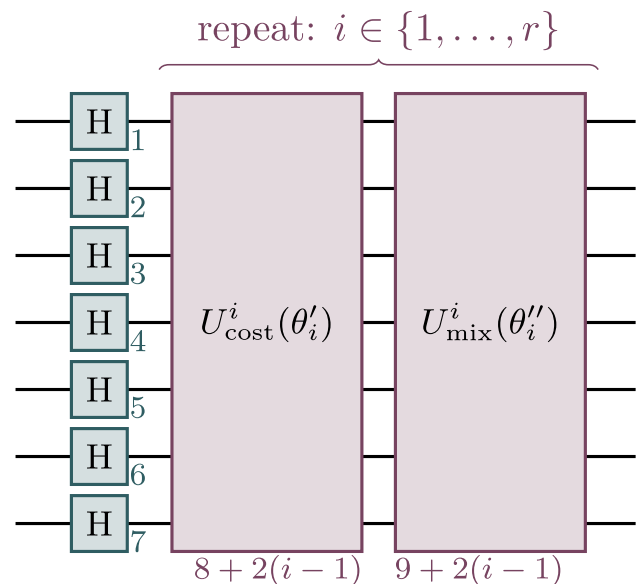


Fig. 22 Circuit for Quantum Approximate Optimization Algorithm QAOA of depth r , which is parameterized by the parameters $\theta \in \mathbb{R}^{2r}$. This circuit contains H gates, a cost layer $U_{\text{cost}}^i(\theta_i')$ depending on the parameter $\theta_i' := \theta_{2(i-1)+1} \in \mathbb{R}$, and a mixing layer $U_{\text{mix}}^i(\theta_i'')$ depending on the parameter $\theta_i'' := \theta_{2(i-1)+2} \in \mathbb{R}$. The number near each gate represents its gate index g , Eq. 22

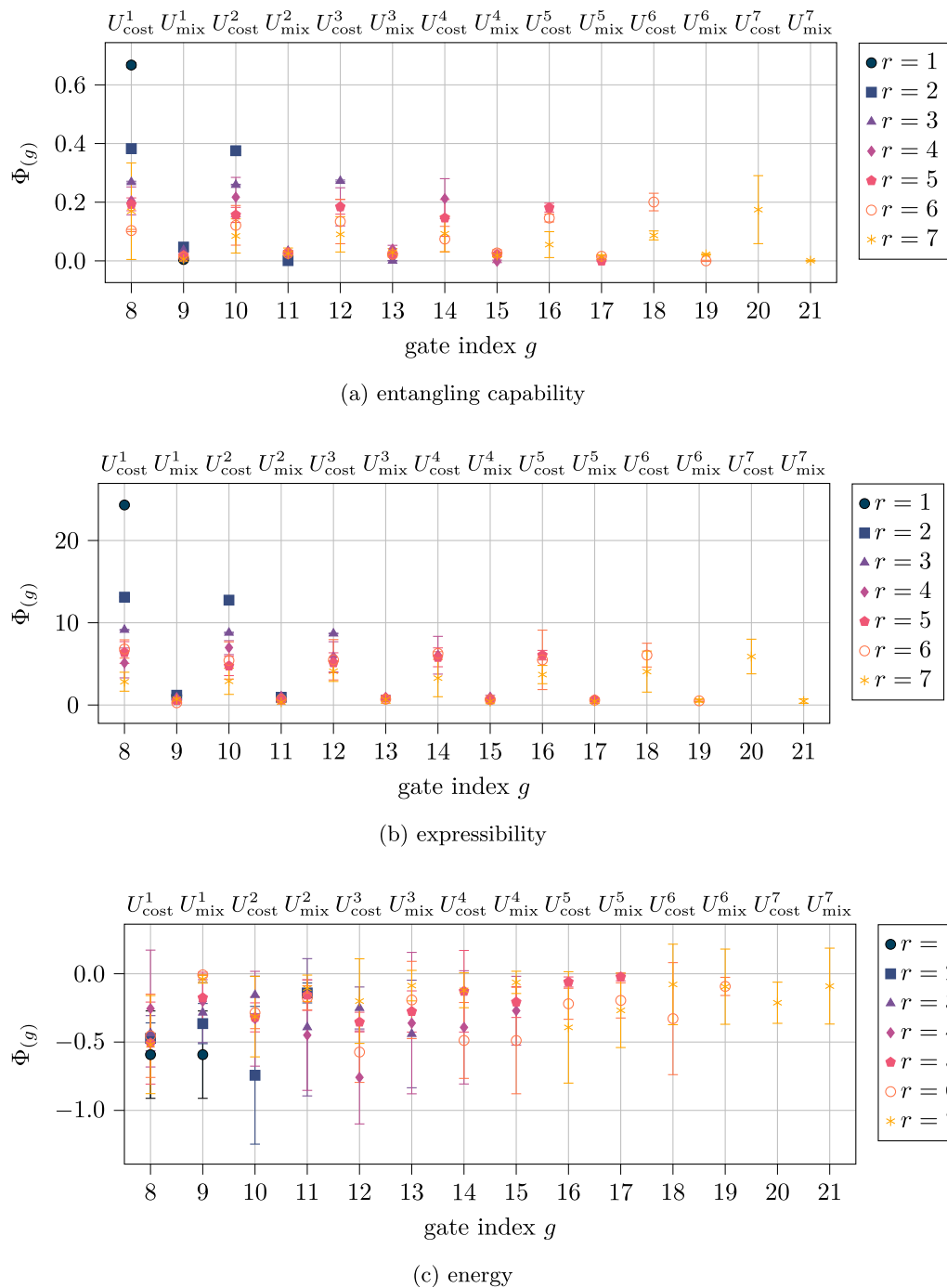


Fig. 23 SVQXs $\Phi(g)$, Eq. 40, for the QAOA circuit of depth $r \in \{1, \dots, 7\}$, Fig. 22, with three value functions of interest: **a** entangling capability, Eq. 37, **b** expressibility, Eq. 36, and **c** energy, Eq. 46. For **a** and **b**, larger SVQXs are more favorable, whereas smaller SVQXs are more favorable for **c**. In analogy to the previous experiments, the gate index g , Eq. 22, is shown on the bottom, whereas the corresponding

gate (i.e., layer) name is shown on the top of the plot. For the gates with $g \in R = \{1, \dots, 7\}$, no SVQX is determined. All evaluations are run on the `state_CPU` simulator. We choose $K = 1$ and $\alpha = 1$ for $r \in \{1, 2, 3\}$, $\alpha = 0.1$ for $r \in \{4, 5\}$, $\alpha = 0.01$ for $r = 6$, and $\alpha = 0.001$ for $r = 7$. For each depth r , we show the mean and one standard deviation over three independent runs

As expected, we find that there is a clear tradeoff between the number of layers and the expressibility and entangling

capability, respectively. For a smaller number of layers, the distribution also becomes significantly broader.

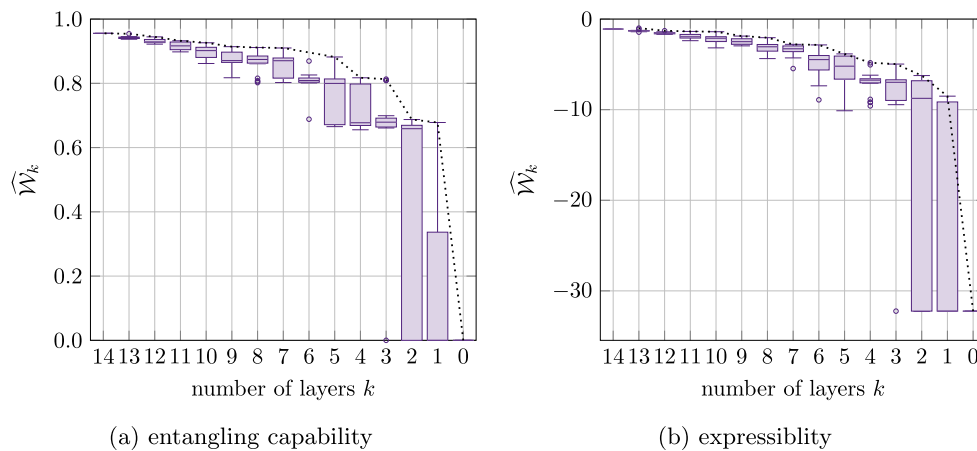


Fig. 24 Distribution of sampled value functions $\hat{W}_k$, Eq. 45, for different number of layers k in form of box plots in analogy to Fig. 7. The results are obtained from one optimization run from Fig. 23 with depth $r = 7$. All evaluations are run on a `state_CPU` simulator with $K = 1$

and $\alpha = 0.001$. The dotted line indicates an approximation of the Pareto frontier for a multi-criteria circuit pruning problem with the goal to minimize the number of layers and to maximize the corresponding value function (expressibility and entangling capability, respectively)

5 Conclusions

In this work, the important XAI tool of SVs has been adopted to explain quantum circuits by analyzing the influence of gates (or groups of gates) on quantum computations. We use the term SVQXs for our approach. While we have mostly focused on its use for approaching model architecture investigations in an XQML context, our method is in fact applicable beyond QML to gate-based quantum computations in general.

The value function for the evaluation of SVQXs can be freely chosen, which allows us to study the impact of quantum gates on a wide range of properties, also within quantum-classical hybrid systems such as a quantum-enhanced ML pipeline. We demonstrate this versatility experimentally with five detailed use cases including QML models for classification and generative modeling as well as quantum optimization tasks with variational quantum circuits. Within these use cases, we perform various explanations with different value functions ranging from model performance over hardware noise and transpilation efficiency to circuit expressibility and entanglement capability. To realize the experiments, we have used both simulators running on a classical host and two different QPUs based on superconducting quantum hardware.

Given this broad applicability, our method opens up opportunities for monitoring and understanding the behavior of real-world quantum computing systems. At the same time, new research questions emerge, e.g., understanding the relation between qubit error metrics or crosstalk (Sarovar et al. 2020) and the context-aware (i.e., gate-specific and circuit-specific) quantities derived via SVQXs to quantify hardware uncertainties for generative models. In addition, our findings also indicate that SVQX could be a promising tool within a QAS framework.

The high complexity, inherited from the classical SVs, is arguably the major limitation of the proposed SVQX approach. A high computational effort is required to cover relevant coalitions and to reduce the uncertainty of the estimated quantities. While our results suggest that an estimation with a subset of coalitions on actual QPUs delivers reasonable results, further research might be required to allow a more general analysis, especially for large scale systems.

Moreover, SVQXs could be modified or extended in many ways as a possible starting point for further studies:

- In place of SVs, Owen values (Owen 1977; López and Saboya 2009) or Banzhaf values (Banzhaf 1965) can be used, concepts that are already used in classical XAI (Chen et al. 2020; Karczmarz et al. 2021).
- Instead of removing gates that are not part of the coalition, other operations could be performed on those gates, e.g., local inversions as proposed recently to quantify the contribution of individual gates to the overall error on NISQ hardware (Calderon-Vargas et al. 2023).
- Certain aspects of XQML like, e.g., an intuitive explanation of quantum feature spaces (Lloyd et al. 2020), go beyond the scope of classical XAI and might require entirely new explainability approaches.

These exemplary aspects indicate that there are still many open research aspects for the explanation of quantum circuits, even within the limited scope of our approach.

So far, XQML is a novel, yet mostly unexplored field that can in the spirit of Schuld and Killoran (2022) be considered as an alternative research direction of QML instead of the search for a quantum advantage. In this sense, we see this contribution as one of the first shots at XQML with a focus on QML model architecture and as a promising cornerstone for further research.

Appendix A: Expressibility

In this appendix section, we briefly present an estimator for the expressibility along the lines of Sim et al. (2019) with the minor extension that we focus on a formulation that can actually be measured on a QPU. For this purpose, we consider an ansatz circuit consisting of q qubits and a corresponding unitary, Eq. 26, that is fully determined by a parameter vector $\vec{\theta} \in \mathcal{P}$, whereas a constant feature vector $\vec{x} \in \mathcal{X}$ (i.e., $\vec{x} = \text{const.}$) is presumed. An estimator of the expressibility of this circuit is given by the Kullback–Leibler divergence

$$\eta(S, R, \vec{x}, c_p, c_b, \mathcal{F}) := D_{\text{KL}}[\hat{P}_{\text{circ}}^{c_p, c_b}(F, S, R, \vec{x}, \mathcal{F}) \| P_{\text{Haar}}(F)]_F \geq 0 \quad (\text{A1})$$

with respect to the fidelity $F \in [0, 1]$.

The first probability distribution in Eq. A1 is an estimation of the probability distribution of fidelities for the circuit of interest. It can be obtained by introducing two i.i.d. random variables $\vec{\theta}_1 \sim u(\mathcal{P})$ and $\vec{\theta}_2 \sim u(\mathcal{P})$, where $u(\mathcal{P})$ denotes the uniform distribution on $\mathcal{P}$. These random variables represent randomly chosen parameter vectors. In total, c_p pairs $\{(\vec{\theta}_1^1, \vec{\theta}_2^1), \dots, (\vec{\theta}_1^{c_p}, \vec{\theta}_2^{c_p})\}$ of those random variables are sampled. For each realization pair $i \in \{1, \dots, c_p\}$, a SWAP test (Barenco et al. 1997; Buhrman et al. 2001) is performed based on s shots.

From these measurements, we obtain the set of bit strings

$$\vec{B}^i(\vec{x}, \vec{\theta}_1^i, \vec{\theta}_2^i) \equiv \vec{B}^i := \{b_1^i, \dots, b_s^i\} \quad (\text{A2})$$

in analogy to Eq. 17 with $b_j^i \in \{0, 1\} \forall j \in \{1, \dots, s\}$. Furthermore, $b_j^i \sim p_{\text{SWAP}}(b_j^i)$ with

$$p_{\text{SWAP}}(b_j^i = 1) = \frac{1}{2} - \frac{1}{2} F^i, \quad (\text{A3})$$

where

$$F^i \equiv F^i(S, R, \vec{x}, \vec{\theta}_1^i, \vec{\theta}_2^i) := |\langle \Psi(S, R, \vec{x}, \vec{\theta}_1^i) | \Psi(S, R, \vec{x}, \vec{\theta}_2^i) \rangle| \in [0, 1] \quad (\text{A4})$$

denotes the fidelity of the two states resulting from the circuit of interest for the randomly sampled parameter vector pair i , where we recall Eq. 16. Thus, an estimate

$$\hat{F}^i := 1 - 2 \frac{|\{b | b \in \vec{B}^i, b = 1\}|}{s} \quad (\text{A5})$$

of F^i can be obtained from the multiset of bit strings.

In total, $s c_p$ shots are necessary to measure the set of fidelities $\mathcal{F} := \{\hat{F}^1, \dots, \hat{F}^{c_p}\}$ for all pairs. From a histogram of $\mathcal{F}$ based on c_b evenly sized bins, the probability distribution

$$\hat{P}_{\text{circ}}^{c_p, c_b}(F, S, R, \vec{x}, \mathcal{F}) := \frac{1}{c_p} \left| \left\{ F' | F' \in \mathcal{F}, \left[\frac{j(c_b, F)}{c_b} \leq F' < \frac{j(c_b, F) + 1}{c_b} \right] \vee [j(c_b, F) + 1 = c_b \wedge F' = 1] \right\} \right|^2 \quad (\text{A6})$$

with

$$j(c_b, F) := \begin{cases} \lfloor F c_b \rfloor & \text{if } 0 \leq F < 1 \\ c_b - 1 & \text{if } F = 1 \end{cases} \quad (\text{A7})$$

can be obtained.

The second probability distribution in Eq. A1 represents the probability density function of fidelities for the ensemble of Haar random states with the analytical form

$$P_{\text{Haar}}(F) := (2^q - 1)(1 - F)^{2^q - 2}, \quad (\text{A8})$$

which only depends on the number of qubits q .

The presented expressibility measure, Eq. A1, is based on a distance measure such that a high value indicates a low expressibility and vice versa. For a more detailed discussion, we refer to Sim et al. (2019); Harrow and Low (2009); Nakaji and Yamamoto (2021) and references therein. An alternative measure can also be found in Du et al. (2022).

Appendix B: Entangling capability

This appendix section outlines a measure for the entangling capability following (Sim et al. 2019). We extend the original concept with a brief description of how it can be measured on a QPU by means of quantum tomography. In analogy to the expressibility calculation in Appendix A, we consider an ansatz circuit consisting of q qubits and a corresponding unitary, Eq. 26, that is fully determined by a parameter vector $\vec{\theta} \in \mathcal{P}$, whereas a constant feature vector $\vec{x} \in \mathcal{X}$ (i.e., $\vec{x} = \text{const.}$) is presumed. A measure for the entangling capability of this circuit is given by the mean

$$\lambda(S, R, \vec{x}, c_s, \mathcal{T}) := \frac{1}{c_s} \sum_{i=1}^{c_s} Q(|\hat{\Psi}^i\rangle) \in [0, 1] \quad (\text{B9})$$

over the Meyer–Wallach entanglement measure (Meyer and Wallach 2002), that is defined as

$$Q(|\Psi\rangle) := \frac{4}{n} \sum_{j=1}^q D(\iota_j(0)|\Psi\rangle, \iota_j(1)|\Psi\rangle) \in [0, 1] \quad (\text{B10})$$

for a q -qubit state $|\Psi\rangle$. Here, we use the mapping

$$\iota_j(b)|b_1 \cdots b_q\rangle := \delta_{bb_j}|b_1 \cdots \hat{b}_j \cdots b_q\rangle \quad (\text{B11})$$

that acts on the computational basis, where $\hat{b}_j$ denotes the absence of the j th qubit and $b \in \{0, 1\}$. Furthermore, we make use of the generalized distance

$$D(|u\rangle, |v\rangle) := \frac{1}{2} \sum_{i,j=1}^{2^q-2} |u_i v_j - u_j v_i|^2 \quad (\text{B12})$$

with $|u\rangle := \sum_{i=1}^{2^q-2} u_i |i\rangle$ and $|v\rangle := \sum_{i=1}^{2^q-2} v_i |i\rangle$, respectively.

In Eq. B9, the Meyer-Wallach entanglement measure is evaluated for all quantum states in $\mathcal{Q} := \{|\hat{\Psi}^1\rangle, \dots, |\hat{\Psi}^{c_s}\rangle\}$. To determine these states, a random variable $\vec{\theta} \sim u(\mathcal{P})$ is introduced and c_s realizations $\mathcal{T} := \{\vec{\theta}^1, \dots, \vec{\theta}^{c_s}\}$ are obtained. For each realization $i \in \{1, \dots, c_s\}$, a quantum state tomography (Zambrano et al. 2020) is performed to obtain

$$|\hat{\Psi}^i\rangle \equiv |\hat{\Psi}^i(S, R, \vec{x}, \vec{\theta}^i, \vec{B}^i)\rangle \approx |\Psi(S, R, \vec{x}, \vec{\theta}^i)\rangle, \quad (\text{B13})$$

where we recall Eq. 26. We presume that a multiset

$$\vec{B}^i \equiv \vec{B}^i(S, R, \vec{x}, \mathcal{T}) := \{b_1^i, \dots, b_s^i\} \quad (\text{B14})$$

of s measurement results in form of bit strings in analogy to Eq. 17 with $b_j^i \in \{0, 1\} \forall j \in \{1, \dots, s\}$ is necessary to achieve this goal, but do not further specify the tomographic process

$$\vec{B}^i \mapsto |\hat{\Psi}^i\rangle, \quad (\text{B15})$$

which can for example correspond to the iterative algorithm presented in Zambrano et al. (2020). In total, $s c_s$ shots are necessary to estimate all quantum states of interest.

According to the Meyer-Wallach entanglement measure, the entangling capability, Eq. B9, becomes larger for more entangled states in $\mathcal{T}$. For a more detailed discussion, we refer to Sim et al. (2019); Meyer and Wallach (2002) and references therein. An alternative interpretation and evaluation of Eq. B9 is presented in Brennen (2003).

Appendix C: Hellinger fidelity

In this appendix section, we briefly outline a metric to quantify NISQ hardware deficiencies. For this purpose, we consider a circuit of interest consisting of q qubits and a corresponding unitary, Eq. 26, that may optionally depend on a parameter vector $\vec{\theta} \in \mathcal{P}$ and feature vector $\vec{x} \in \mathcal{X}$, respectively, which are both presumed to be constant (i.e., $\vec{\theta} = \text{const.}$ and $\vec{x} = \text{const.}$). The circuit is evaluated independently with two different approaches. First, on a QPU to

obtain the measurement results $\vec{B}(S)$, Eq. 27, as a collection of bit strings. And second, with an idealized (i.e., noise-free) simulator running on classical hardware.

The estimated probability distribution of bit strings $\vec{b} \in \{0, 1\}^q$ that can be obtained from the QPU is given by

$$m_{\text{qc}}(\vec{b}, \vec{B}(S)) := \frac{|\{\vec{b}' \mid \vec{b}' \in \vec{B}(S), \vec{b}' = \vec{b}\}|}{s} \quad (\text{C16})$$

for s shots. On the other hand, the corresponding probability distribution obtained from the classical simulator is given by $m_{\text{sim}}(\vec{b}) \equiv m_{\text{sim}}(\vec{b}, S, R, \vec{x}, \vec{\theta})$ according to Eq. 28. The Hellinger fidelity (Hellinger 1909)

$$H(S, R, \vec{x}, \vec{\theta}, \vec{B}(S)) := \left[\sum_{\vec{b} \in \{0, 1\}^q} \sqrt{m_{\text{qc}}(\vec{b}, \vec{B}(S)) m_{\text{sim}}(\vec{b})} \right]^2 \in [0, 1] \quad (\text{C17})$$

is a similarity measure of the two distributions. It can therefore be used as a metric to quantify NISQ hardware deficiencies. By definition, a smaller fidelity indicates larger hardware deficiencies.

Appendix D: Estimated execution efficiency

This appendix section considers the estimated execution efficiency as a measure of how efficiently a circuit can be executed on a specific NISQ hardware device. The challenge of efficient execution arises because of two typical limitations of NISQ hardware. First, only a limited set of gates can be physically realized on a hardware device. However, presuming that this set of available gates is a universal set of quantum gates, all other gates can be expressed in terms of the available gates (Nielsen et al. 2011; Tucci 2005). Second, the connectivity of physical qubits in a hardware device is limited in the sense that gates acting on two (or more) qubits can only be applied on certain pairs (or sets) of qubits. However, this limitation can be overcome by introducing additional gates to swap qubits accordingly (Nielsen et al. 2011). Due to these two limitations, any quantum circuit must be subjected to a suitable equivalence transformation before execution such that it contains only gates from the set of available gates $\mathcal{U}$ as defined by the hardware device of interest. This equivalence transformation is also called transpilation (Anis et al. 2019).

Specifically, we consider a transpilation procedure T as an equivalence transformation of a gate set $P(S, R) := \{A_a \mid a \in S\} \cup R$ from Eq. 25 into a new gate set $P'(S, R) := \{1, \dots, G'\} \cup R$ according to

$$T : P(S, R) \mapsto P'(S, R) \quad (\text{D18})$$

such that

$$U(S, R, \vec{x}, \vec{\theta}) = U'(S, R, \vec{x}, \vec{\theta}), \quad (\text{D19})$$

where

$$U'(S, R, \vec{x}, \vec{\theta}) := \prod_{g \in P'(S, R)} U_g(\vec{x}^g, \vec{\theta}^g) \quad (\text{D20})$$

in analogy to Eq. 25 with unitary operators $U'_g(\vec{x}^g, \vec{\theta}^g) \in \mathcal{U}$ for all $g \in P'(S, R)$. The set $\mathcal{U}$ contains all gates that are available on the hardware device of interest. For the sake of simplicity, we presume that $\mathcal{U}$ contains only one-qubit and two-qubit gates.

Generally, the equivalence transformation of a circuit is ambiguous and can therefore lead to different gate decompositions with a varying number of gates. Since every gate introduces hardware-related uncertainty into the corresponding QPU computation, an efficient transpilation with at least gates as possible in the resulting circuit is desired, which is a non-trivial optimization task. The estimated transpilation efficiency

$$\tau_T(S, R, \mathcal{U}, s_1, s_2) := \sum_{g \in P'(S, R)} \hat{\tau}(U'_g(\vec{x}^g, \vec{\theta}^g), s_1, s_2) \leq 0 \quad (\text{D21})$$

is a possible metric to measure the quality of a certain transpilation T for an available gate set $\mathcal{U}$ based on the resulting gate decomposition. The penalty function

$$\hat{\tau}(U, s_1, s_2) := \begin{cases} s_2 & \text{if } U \in \mathcal{U}_2 \\ s_1 & \text{otherwise} \end{cases} \quad (\text{D22})$$

estimates the decrease in efficiency for a higher number of gates by assigning a negative penalty value to each gate,

where $\mathcal{U}_2 \subset \mathcal{U}$ contains all available two-qubit gates. Hence, a higher penalty score indicates a higher estimated transpilation efficiency (since less noise can be expected). The penalty parameters $s_1, s_2 \in \mathbb{R}_{<0}$ can be chosen arbitrarily, where typically $0 > s_1 > s_2$ to reflect that two-qubit gates induce more noise than one-qubit gates.

The estimated execution efficiency

$$\tau(S, R, \mathcal{U}, s_1, s_2) := \max_T \tau_T(S, R, \mathcal{U}, s_1, s_2) \leq 0 \quad (\text{D23})$$

is defined as the largest estimated transpilation efficiency, Eq. D21, for all possible transpilation procedures. Summarized, the estimated execution efficiency represents the largest possible sum over the penalty scores of all gates and can therefore be used to estimate how efficiently the corresponding circuit can be executed on the hardware device of interest.

More generally, the efficiency of a transpilation procedure may not only depend on the absolute number of gates, but also the noise properties of individual qubits and gates as well as low-level hardware realizations to achieve a better performance (Tannu et al. 2018; Murali et al. 2019; Wilson et al. 2020; Earnest et al. 2021; Nation et al. 2022). We do not consider this more complex case here, which can, however, be brought to the same formalism by defining a suitably modified penalty function.

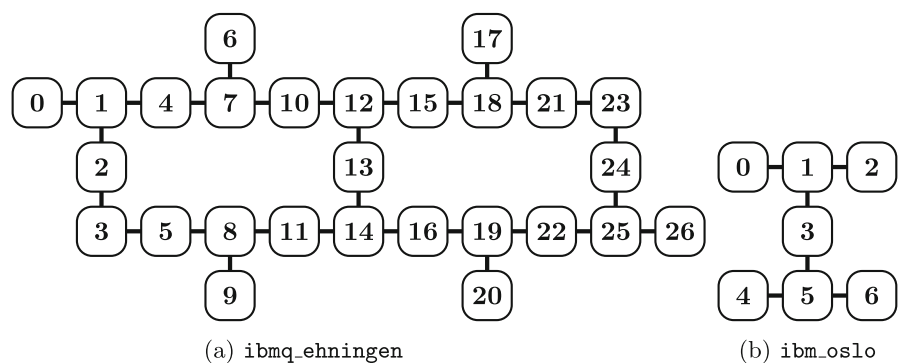
Appendix E: NISQ devices

We use two NISQ devices to perform our experiments: `ibmq_ehningen` (accessible via IBM 2022a) and

Table 1 NISQ device specifications as provided by IBM

QPU	Processor type	Version	Number of qubits	Quantum volume (Cross et al. 2019)	CLOPS (Wack et al. 2021)
<code>ibmq_ehningen</code>	Falcon r5.11	3.1.22	27	64	1.9×10^3
<code>ibmq_oslo</code>	Falcon r5.11H	1.0.14	7	32	2.6×10^3
QPU	median CX error	median readout error	median T_1	median T_2	reference
<code>ibmq_ehningen</code>	7.218×10^{-3}	9.300×10^{-3}	127.09 μs	143.27 μs	IBM (2022a)
<code>ibmq_oslo</code>	9.748×10^{-3}	1.950×10^{-2}	148.03 μs	41.79 μs	IBM (2022b)

Fig. 25 Hardware connectivity map for the two NISQ devices of interest. We show the physical qubit indices (unique identifiers) and connections between qubits that allow the realization of joint CX gates



`ibm_oslo` (accessible via IBM 2022b). The specifications of these devices—as provided by IBM—are listed in Table 1. Both QPUs can realize four elementary gates that form a universal gate set: (i) three one-qubit gates, namely the Z-rotation gate $R_Z(\lambda)$ (parameterized by an angle λ), the NOT gate X, and the SX gate $\sqrt{X}$, as well as (ii) the two-qubit controlled NOT gate CX. In Fig. 25, we show the hardware connectivity maps. Each node represents a qubit labeled with the corresponding physical qubit index (that allows a unique

identification) and each edge indicate the respective qubit connections that allow the realization of joint CX gates.

Appendix F: Circuit symbols

Throughout this paper, we use the circuit symbols shown in Fig. 26 to denote quantum gates. All gates are used as defined by Qiskit (Anis et al. 2019).

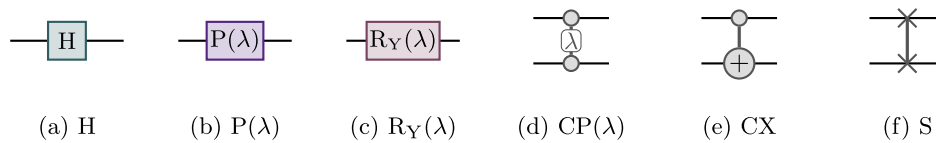


Fig. 26 Overview over the circuit symbols we use throughout the paper: Hadamard gate H, phase gate $P(\lambda)$, Y-rotation gate $R_Y(\lambda)$, controlled phase gate $CP(\lambda)$, controlled NOT gate CX, and SWAP gate S gate. Here, λ denotes a real parameter

Appendix G: QSVM marginal contributions

In this appendix section, we provide supplementary material for the QSVM use case from Sect. 4.1. Specifically, we investigate the distribution of marginal contributions $p_i(\Delta_i v)$, Eq. 9, for each player index i , where i corresponds to the gate index g . The results are based on the evaluation from the `state_CPU` simulator with $K = 1$. For $r \in \{1, 2\}$, we use $\alpha = 1$, whereas for $r = 3$, we use $\alpha = 0.01$ with a single run. That is, we use the same setup as for Fig. 6, but consider only the first run for $r = 3$.

The results are shown in Fig. 27. Each circle represents a marginal contribution with a size that monotonically depends on the corresponding probability (i.e., the larger, the more probable). In addition, we also show the mean, Eq. 8, and one standard deviation, Eq. 12, for each index i . For the sake of simplicity, we omit values with $p < 0.001$. Furthermore, we renormalize the probabilities to one for $r = 3$ (which are not normalized due to the subsampling).

By definition, the mean values from Fig. 27 correspond to the respective SVQXs from Fig. 6. We find that almost all SVQXs are within one standard deviation of the other SVQXs.

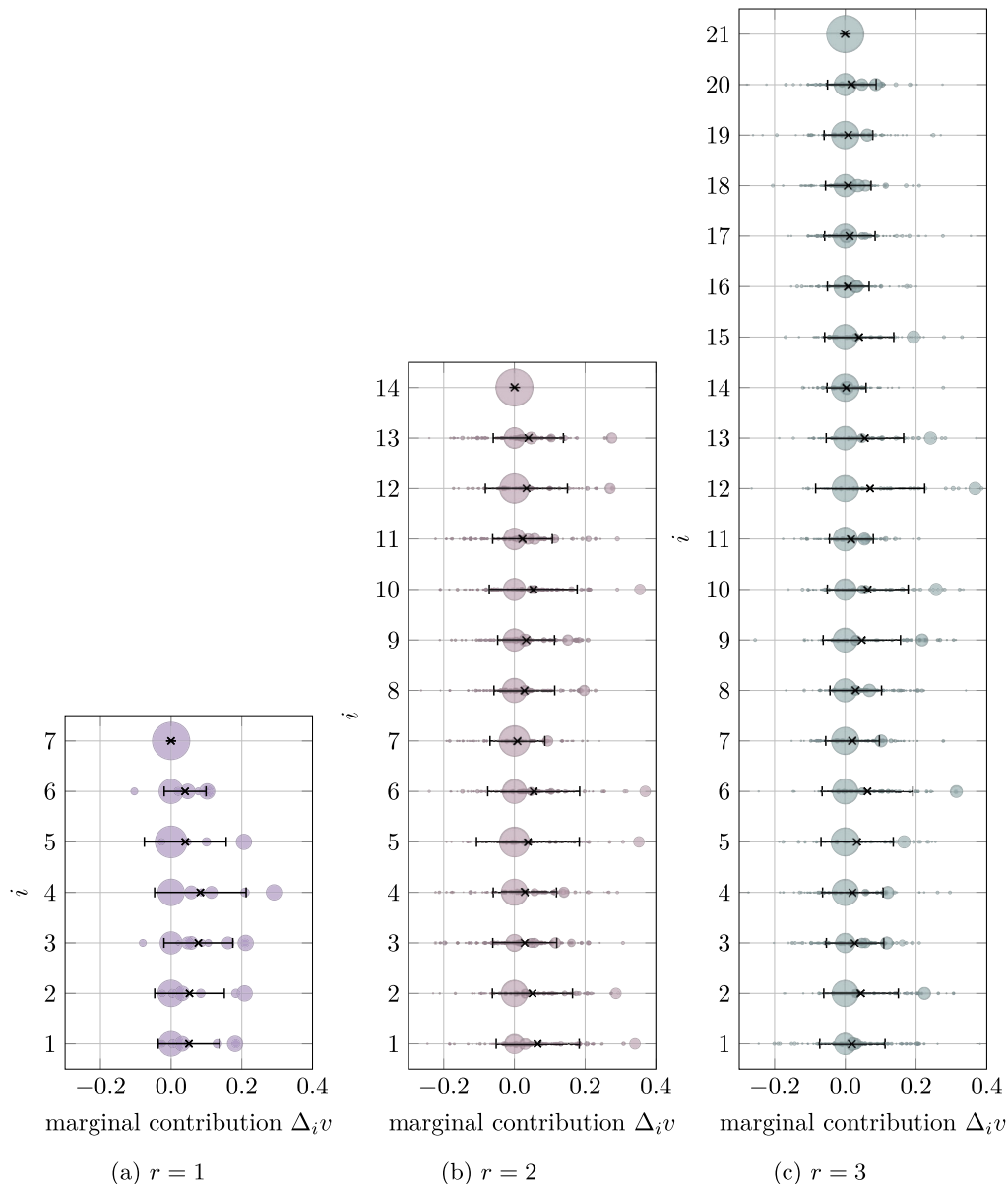


Fig. 27 Distribution of marginal contributions $p_i(\Delta_i v)$, Eq. 9, for the QSVM use case from Sect. 4.1 with repetition numbers $r \in \{1, 2, 3\}$. For each player index i , we show the marginal contributions as circles with a size that monotonically depends on the corresponding proba-

bility. All evaluations are based on the `state_CPU` simulator with $K = 1$. We use $\alpha = 1$ for $r \in \{1, 2\}$ and $\alpha = 0.01$ for $r = 3$. For each player index i , we show the mean, Eq. 8, and one standard deviation, Eq. 12

Appendix H: QGAN timing of experiments

The start and end times for the QGAN experiments from Sect. 4.3 are presented in Table 2. Here, the configurations and runs refer to the results shown in Fig. 17.

Table 2 Timing of the runs from Fig. 17

Configuration	Run	Start time	End time	Samples
0-1-2	1	2022-12-10 16:01:01	2022-12-11 19:12:15	28
	2	2022-12-11 19:12:15	2022-12-12 18:54:38	24
	3	2022-12-12 18:54:39	2022-12-13 23:51:04	29
0-1-2 with MEM	1	2022-12-10 16:01:21	2022-12-11 20:23:36	29
	2	2022-12-11 20:23:36	2022-12-13 07:14:06	35
	3	2022-12-13 07:14:07	2022-12-14 11:24:05	29
3-5-4	1	2022-12-21 12:39:12	2022-12-23 02:51:03	39
	2	2022-12-23 02:51:03	2022-12-24 22:27:32	44
	3	2022-12-24 22:27:33	2022-12-26 05:47:31	32
3-5-4 with MEM	1	2022-12-21 12:40:53	2022-12-23 08:54:24	45
	2	2022-12-23 08:54:24	2022-12-25 03:22:59	43
	3	2022-12-25 03:22:59	2022-12-26 07:15:21	28

For each experiment and each run, we present the start time, the end time and the number of hourly collected error samples between start and end time as described in Sect. 4.3. Each sample consists of a single gate error and a CX error, respectively, for each qubit or qubit pair of interest. The time intervals include waiting times and the execution of MEM. All dates are in the format *YYYY-MM-DD HH:MM:SS*

Appendix I: QAOA optimization results

The optimization results for the max-cut problem from Sect. 4.5 are listed in Table 3. Specifically, we consider three independent runs for different circuit depths $r \in \{1, \dots, 7\}$, each with a different random seed for the classical optimizer. The global optimum has a value of 9.

Table 3 Optimization goals (i.e., number of cuts in the graph from Fig. 21) resulting from Quantum Approximate Optimization Algorithm QAOA for different depths r with three independent runs per depth

r	Optimization goal		
	run 1	run 2	run 3
1	9	9	6
2	9	9	6
3	6	9	9
4	9	9	9
5	9	9	9
6	9	9	9
7	9	9	9

The global optimum is highlighted in bold. All calculations are performed on a state_CPU simulator

Acknowledgements Parts of this research have been funded by the Ministry of Science and Health of the State of Rhineland-Palatinate as part of the project *AnQuC*. Parts of this research have been funded by the Federal Ministry of Education and Research of Germany and the state of North-Rhine Westphalia as part of the *Lamarr-Institute for Machine Learning and Artificial Intelligence*. Access to quantum computing hardware has been funded by *Fraunhofer Quantum Now*.

Author contribution All authors contributed to the writing and overall research process. R.H., S.M., M.J., and N.P. were responsible for the conceptualization. R.H., T.G., and S.Mc. wrote the software. R.H. developed the methodology. R.H. and T.G. performed the investigations. R.H. prepared the figures. N.P. was responsible for project administration. All authors reviewed the manuscript.

Funding Open Access funding enabled and organized by Projekt DEAL. Parts of this research have been funded by the Ministry of Science and Health of the State of Rhineland-Palatinate as part of the project *AnQuC*. Parts of this research have been funded by the Federal Ministry of Education and Research of Germany and the state of North-Rhine Westphalia as part of the *Lamarr-Institute for Machine Learning and Artificial Intelligence*. Access to quantum computing hardware has been funded by *Fraunhofer Quantum Now*.

Data availability All data is publicly available online via the respective references.

Declarations

Conflict of interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Aha DW (1991) Tic-Tac-Toe endgame data set. <https://archive.ics.uci.edu/ml/datasets/Tic-Tac-Toe+Endgame>
- Anis MS, et al (2019) Qiskit: an open-source framework for quantum computing
- Aumann RJ (1994) Economic applications of the Shapley value. Springer Netherlands, Dordrecht, pp 121–133. https://doi.org/10.1007/978-94-017-1656-7_12
- Aumann RJ, Shapley LS (1974) Values of non-atomic games, Princeton University Press. <http://www.jstor.org/stable/j.ctt13x149m>
- Baheri B, Guan Q, Chaudhary V, Li A, Savino A, Rech P, Carlo SD, Gizopoulos D (2022) Quantum noise in the flow of time: a temporal study of the noise in quantum computers. In: Savino A, Rech P, Carlo SD, Gizopoulos D (eds) 2022 IEEE 28th international symposium on on-line testing and robust system design (IOLTS), pp 1–5
- Banzhaf J (1965) Weighted voting doesn't work: a mathematical analysis. *Rutgers Law Rev* 19:317–343
- Barbudo R, Ventura S, Romero JR (2023) Eight years of automl: categorisation, review and trends. *Knowl Inf Syst* 65:5097–5149. <https://doi.org/10.1007/s10115-023-01935-1>
- Barenco A et al (1997) Stabilization of quantum computations by symmetrization. *SIAM J Comput* 26:1541–1557. <https://doi.org/10.1137/S0097539796302452>
- Biamonte J et al (2017) Quantum machine learning. *Nature* 549:195–202. <https://doi.org/10.1038/nature23474>
- Brennen GK (2003) An observable measure of entanglement for pure states of multi-qubit systems. <https://arxiv.org/abs/quant-ph/0305094>
- Buhrman H, Cleve R, Watrous J, de Wolf R (2001) Quantum fingerprinting. *Phys Rev Lett* 87:167902. <https://doi.org/10.1103/PhysRevLett.87.167902>
- Burge I, Barbeau M, Garcia-Alfaro J (2024) A Shapley value estimation speedup for efficient explainable quantum ai. [arXiv:2412.14639](https://arxiv.org/abs/2412.14639)
- Burnett JJ et al (2019) Decoherence benchmarking of superconducting qubits. *NPJ Quantum Inf* 5:54. <https://doi.org/10.1038/s41534-019-0168-5>
- Calderon-Vargas FA, Proctor T, Rudinger K, Sarovar M (2023) Quantum circuit debugging and sensitivity analysis via local inversions. *Quantum* 7:921. <https://doi.org/10.22331/q-2023-02-09-921>
- Caro MC et al (2022) Generalization in quantum machine learning from few training data. *Nat Commun* 13:1–11. <https://doi.org/10.1038/s41467-022-32550-3>
- Cerezo M et al (2021) Variational quantum algorithms. *Nat Rev Phys* 3:625–644. <https://doi.org/10.1038/s42254-021-00348-9>
- Chen SY-C (2024) Differentiable quantum architecture search in asynchronous quantum reinforcement learning. [arXiv:2407.18202](https://arxiv.org/abs/2407.18202)
- Chen H, Zheng G, Ji Y (2020) Generating hierarchical explanations on text classification via feature interaction detection 2004:02015

- Ciaramella G, Martinelli F, Mercaldo F, Santone A, IJCNN (2023) Exploring quantum machine learning for explainable malware detection. In: 2023 International Joint Conference on Neural Networks (IJCNN). IEEE, pp 1–6
- Coppersmith D (2002) An approximate Fourier transform useful in quantum factoring. *arXiv:0201067*
- Cortes C, Vapnik V (1995) Support-vector networks. *Mach Learn* 20:273–297. <https://doi.org/10.1007/BF00994018>
- Cross AW, Bishop LS, Sheldon S, Nation PD, Gambetta JM (2019) Validating quantum computers using randomized model circuits. *Phys Rev A* 100:032328. <https://doi.org/10.1103/PhysRevA.100.032328>
- Cuellar MP, Pegalajar MC, Cano C (2024) Automatic evolutionary design of quantum rule-based systems and applications to quantum reinforcement learning. *Quantum Inf Process* 23:179. <https://doi.org/10.1007/s11128-024-04391-0>
- Diaz-Ortiz Jr M, Kempinski B, Cornelisse D, Bachrach Y, Kachman T (2023) Using cooperative game theory to prune neural networks. *arXiv:2311.10468*
- Ding L, Spector L (2022) Evolutionary quantum architecture search for parametrized quantum circuits. In: GECCO (ed) Proceedings of the genetic and evolutionary computation conference companion, GECCO '22, 2190–2195. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3520304.3534012>
- Du Y, Huang T, You S, Hsieh M-H, Tao D (2022) Quantum circuit architecture search for variational quantum algorithms. *NPJ Quantum Inf* 8:62. <https://doi.org/10.1038/s41534-022-00570-y>
- Dunjko V, Taylor JM, Briegel HJ (2016) Quantum-enhanced machine learning. *Phys Rev Lett* 117. <https://doi.org/10.1103/PhysRevLett.117.130501>
- Du Y, Tu Z, Yuan X, Tao D (2022) Efficient measure for the expressivity of variational quantum algorithms. *Phys Rev Lett* 128. <https://doi.org/10.1103/PhysRevLett.128.080506>
- Earnest N, Tornow C, Egger DJ (2021) Pulse-efficient circuit transpilation for quantum applications on cross-resonance-based hardware. *Phys Rev Res* 3. <https://doi.org/10.1103/PhysRevResearch.3.043088>
- Farhi E, Goldstone J, Gutmann S (2014) A quantum approximate optimization algorithm. *arXiv:1411.4028*
- Fatima SS, Wooldridge M, Jennings NR (2007) A randomized method for the Shapley value for the voting game. In: AAMAS (ed) Proceedings of the 6th international joint conference on autonomous agents and multiagent systems, AAMAS '07. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/1329125.1329316>
- Flamini F, Krumm M, Fiderer LJ, Müller T, Briegel HJ (2024) Towards interpretable quantum machine learning via single-photon quantum walks. *Quantum Sci Technol* 9:045011. <https://doi.org/10.1088/2058-9565/ad5907>
- Franken L, et al (2022) Quantum circuit evolution on NISQ devices. In: CEC (ed) 2022 IEEE congress on evolutionary computation (CEC). IEEE, pp 1–8. <https://doi.org/10.1109/CEC55065.2022.9870269>
- Gambetta J (2022) Expanding the IBM Quantum roadmap to anticipate the future of quantum-centric supercomputing. <https://research.ibm.com/blog/ibm-quantum-roadmap-2025>
- Gentinetta G, Thomsen A, Sutter D, Woerner S (2022) The complexity of quantum support vector machines. *arXiv:2203.00031*
- Georgopoulos K, Emary C, Zuliani P (2021) Modeling and simulating the noisy behavior of near-term quantum computers. *Phys Rev A* 104. <https://doi.org/10.1103/PhysRevA.104.062432>
- Gil-Fuster E, et al (2024) Opportunities and limitations of explaining quantum machine learning. *arXiv:2412.14753*
- Giovagnoli A, Tresp V, Ma Y, Schubert M (2023) QNEAT: natural evolution of variational quantum circuit architecture. In: GECCO (ed) Proceedings of the companion conference on genetic and evolutionary computation, GECCO '23 Companion. Association for Computing Machinery, New York, NY, USA, pp 647–650. <https://doi.org/10.1145/3583133.3590675>
- Gujju Y, Matsuo A, Raymond R (2024) Quantum machine learning on near-term quantum devices: current state of supervised and unsupervised techniques for real-world applications. *Phys Rev Appl* 21:067001. <https://link.aps.org/doi/10.1103/PhysRevApplied.21.067001>
- Hadfield S et al (2019) From the quantum approximate optimization algorithm to a quantum alternating operator ansatz. *Algorithms* 12:34. <https://doi.org/10.3390/a12020034>
- Harrow AW, Low RA (2009) Random quantum circuits are approximate 2-designs. *Commun Math Phys* 291:257–302. <https://doi.org/10.1007/s00220-009-0873-6>
- Haug T, Bharti K, Kim M (2021) Capacity and quantum geometry of parametrized quantum circuits. *PRX Quantum* 2. <https://doi.org/10.1103/PRXQuantum.2.040309>
- Havlíček V et al (2019) Supervised learning with quantum-enhanced feature spaces. *Nature* 567:209–212. <https://doi.org/10.1038/s41586-019-0980-2>
- Heese R (2022) Quantum decision trees (qtree). <https://github.com/RaoulHeese/qtree>
- Heese R (2023) Quantum Shapley value toolbox. <https://github.com/RaoulHeese/qshaptools>
- Heese R, Bickert P, Niederle AE (2022) Representation of binary classification trees with binary features by quantum circuits. *Quantum* 6:676. <https://doi.org/10.22331/q-2022-03-30-676>
- Heese R, et al (2023) Data for: explainable quantum machine learning. <https://doi.org/10.5281/zenodo.7603939>
- Heese R, Mücke S, Jakobs M, Gerlach T, Piatkowski N (2023) Shapley values with uncertain value functions. *arXiv:2301.08086*
- Hellinger E (1909) Neue Begründung der Theorie quadratischer Formen von unendlichvielen Veränderlichen. *Journal für die reine und angewandte Mathematik* 1909:210–271. <https://doi.org/10.1515/crll.1909.136.210>
- Holmes Z, Sharma K, Cerezo M, Coles PJ (2022) Connecting ansatz expressibility to gradient magnitudes and barren plateaus. *PRX Quantum* 3. <https://doi.org/10.1103/PRXQuantum.3.010313>
- Huang H-Y et al (2021) Power of data in quantum machine learning. *Nat Commun* 12:1–9. <https://doi.org/10.1038/s41467-021-22539-9>
- Huang X, Marques-Silva J (2023) The inadequacy of Shapley values for explainability. *arXiv:2302.08160*
- IBM (2022a) IBM quantum Germany. <https://de.quantum-computing.ibm.com>
- IBM (2022b) IBM quantum. <https://quantum-computing.ibm.com>
- Karczmarz A, Mukherjee A, Sankowski P, Wygocki P (2021) Improved feature importance computations for tree models: Shapley vs. Banzhaf. *arXiv:2108.04126*
- Khan FS, Solmeyer N, Balu R, Humble TS (2018) Quantum games: a review of the history, current state, and interpretation. *Quantum Inf Process* 17. <https://doi.org/10.1007/s11128-018-2082-8>
- Kiani BT, Palma GD, Marvian M, Liu Z-W, Lloyd S (2022) Learning quantum data with the quantum earth mover's distance. *Quantum Sci Technol* 7:045002. <https://doi.org/10.1088/2058-9565/ac79c9>
- Klau D, Zöller M, Tutschku C (2023) Bringing quantum algorithms to automated machine learning: a systematic review of automl frameworks regarding extensibility for qml algorithms. *arXiv:2310.04238*
- Kumar IE, Venkatasubramanian S, Scheidegger C, Friedler S, III HD, Singh A (2020) Problems with Shapley-value-based explanations as feature importance measures. In: III HD, Singh A (eds) Proceedings of the 37th international conference on machine learning, Vol. 119 of Proceedings of Machine Learning Research. PMLR, pp 5491–5500. <https://proceedings.mlr.press/v119/kumar20e.html>
- Kyriienko O, Magnusson EB (2022) Unsupervised quantum machine learning for fraud detection 2208:01203

- Leon F (2014) Optimizing neural network topology using Shapley value. In: ICSTCC (ed) 2014 18th international conference on system theory, control and computing (ICSTCC), pp 862–867
- Lin H-Y, Tseng H-H, Chen SY-C, Yoo S, Gurrola J (2024) Quantum gradient class activation map for model interpretability. In: Gurrola J (ed), 2024 IEEE workshop on signal processing systems (SIPS). IEEE, pp 165–170
- Linardatos P, Papastefanopoulos V, Kotsiantis S (2020) Explainable ai: a review of machine learning interpretability methods. *Entropy* 23. <https://www.mdpi.com/1099-4300/23/1/18>
- Liu Y, Arunachalam S, Temme K (2021) A rigorous and robust quantum speed-up in supervised machine learning. *Nat Phys* 17:1013–1017. <https://doi.org/10.1038/s41567-021-01287-z>
- Lloyd S, Schuld M, Ijaz A, Izaac J, Killoran N (2020) Quantum embeddings for machine learning. [arXiv:2001.03622](https://arxiv.org/abs/2001.03622)
- López S, Saboya M (2009) On the relationship between Shapley and Owen values. *Cent Eur J Oper Res* 17:415–423. <https://doi.org/10.1007/s10100-009-0100-8>
- Lundberg SM, Lee S-I (2017) A unified approach to interpreting model predictions. In: Guyon I, et al (eds) *Advances in neural information processing systems*, vol 30, pp 4765–4774. <http://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf>
- Marcinkevičs R, Vogt JE (2020) Interpretability and explainability: a machine learning zoo mini-tour. <https://arxiv.org/abs/2012.01805>
- Maria Schuld FP (2018) *Supervised learning with quantum computers 1 edn*. Quantum Science and Technology, Springer Cham
- Mercaldo F, et al (2022) Towards explainable quantum machine learning for mobile malware detection and classification. *Appl Sci* 12. <https://www.mdpi.com/2076-3417/12/23/12025>
- Meyer N et al (2022) A survey on quantum reinforcement learning 2211:03464
- Meyer DA, Wallach NR (2002) Global entanglement in multiparticle systems. *J Math Phys* 43:4273–4278. <https://doi.org/10.1063/1.1497700>
- Minh D, Wang HX, Li YF, Nguyen TN (2022) Explainable artificial intelligence: a comprehensive review. *Artif Intell Rev* 55:3503–3568. <https://doi.org/10.1007/s10462-021-10088-y>
- Moll N et al (2018) Quantum optimization using variational algorithms on near-term quantum devices. *Quantum Sci Technol* 3:030503. <https://doi.org/10.1088/2058-9565/aab822>
- Molnar C (2022) *Interpretable machine learning 2 edn*. <https://christophm.github.io/interpretable-ml-book>
- Mukhamediev RI, et al (2022) Review of artificial intelligence and machine learning technologies: classification, restrictions, opportunities and challenges. *Mathematics* 10. <https://www.mdpi.com/2227-7390/10/15/2552>
- Murali P, Baker JM, Abhari AJ, Chong FT, Martonosi M (2019) Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers. [arXiv:1901.11054](https://arxiv.org/abs/1901.11054)
- Nakaji K, Yamamoto N (2021) Expressibility of the alternating layered ansatz for quantum computation. *Quantum* 5:434. <https://doi.org/10.22331/q-2021-04-19-434>
- Nation PD, Kang H, Sundaresan N, Gambetta JM (2021) Scalable mitigation of measurement errors on quantum computers. *PRX Quantum* 2. <https://doi.org/10.1103/PRXQuantum.2.040326>
- Nation P, Treinish M, Possel C (2022) Mapomatic. <https://github.com/Qiskit-Partners/mapomatic>
- Nielsen MA, Chuang IL (2011) *Quantum computation and quantum information: 10th anniversary edition, 10th edn*. Cambridge University Press, USA
- Oloulade BM et al (2025) Shapley-guided pruning for efficient graph neural architecture prediction in distributed learning environments. *Inf Sci* 695:121695. <https://www.sciencedirect.com/science/article/pii/S0020025524016098>
- Owen G, Henn R, Moeschlin O (1977) Values of games with a priori unions. In: Henn R, Moeschlin O (eds) *Mathematical economics and game theory*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp 76–88
- Pira L, Ferrie C (2024) On the interpretability of quantum neural networks. *Quantum Mach Intell* 6:52. <https://doi.org/10.1007/s42484-024-00191-y>
- Powell MJD (2007) A view of algorithms for optimization without derivatives. Tech. Rep. 5, Institute of Mathematics and its Applications
- Power L, Guha K (2024) Feature importance and explainability in quantum machine learning. [arXiv:2405.08917](https://arxiv.org/abs/2405.08917)
- Preskill J (2018) Quantum computing in the NISQ era and beyond. *Quantum* 2:79. <https://doi.org/10.22331/q-2018-08-06-79>
- Rozemberczki B, et al (2022) The Shapley value in machine learning. [arXiv:2202.05594](https://arxiv.org/abs/2202.05594)
- Ruan S et al (2024) Violet: visual analytics for explainable quantum neural networks. *IEEE Trans Vis Comput Graph* 30:2862–2874
- Rybotycki T, Gawron P (2024) Aqmlator – an auto quantum machine learning e-platform. [arXiv:2409.18338](https://arxiv.org/abs/2409.18338)
- Saeed W, Omlin C (2023) Explainable AI (XAI): a systematic meta-survey of current challenges and future opportunities. *Knowl Based Syst* 263:110273. <https://www.sciencedirect.com/science/article/pii/S0950705123000230>
- Salehin I et al (2024) AutoML: a systematic review on automated machine learning with neural architecture search. *J Inf Intell* 2:52–81. <https://www.sciencedirect.com/science/article/pii/S2949715923000604>
- Salmani Pour Avval S, Eskue ND, Groves RM, Yaghoubi V (2025) Systematic review on neural architecture search. *Artif Intell Rev* 58:73. <https://doi.org/10.1007/s10462-024-11058-w>
- Sarovar M et al (2020) Detecting crosstalk errors in quantum information processors. *Quantum* 4:321. <https://doi.org/10.22331/q-2020-09-11-321>
- Schuld M (2021) Supervised quantum machine learning models are kernel methods 2101:11020
- Schuld M, Killoran N (2022) Is quantum advantage the right goal for quantum machine learning?. [arXiv:2203.01340](https://arxiv.org/abs/2203.01340)
- Schuld M, Sweke R, Meyer JJ (2021) Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Phys Rev A* 103. <https://doi.org/10.1103/PhysRevA.103.032430>
- Shapley LS (1953) A value for n-person games. *Contributions to the Theory of Games* 2:307–317
- Sim S, Johnson PD, Aspuru-Guzik A (2019) Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms. *Ad Quantum Technol* 2:1900070. <https://doi.org/10.1002/qute.201900070>
- Sim S, Romero J, Gonthier JF, Kunitsa AA (2021) Adaptive pruning-based optimization of parameterized quantum circuits. *Quantum Sci Technol* 6:025019. <https://doi.org/10.1088/2058-9565/abe107>
- Srivastava N, Hinton G, Krizhevsky A, Sutskever I, Salakhutdinov R (2014) Dropout: a simple way to prevent neural networks from overfitting. *J Mach Learn Res* 15:1929–1958. <http://jmlr.org/papers/v15/srivastava14a.html>
- Stier J, Gianini G, Granitzer M, Ziegler K (2018) Analysing neural network topologies: a game theoretic approach. *Proc Comput Sci* 126:234–243. <https://www.sciencedirect.com/science/article/pii/S187705091831233X>. Knowledge-Based and Intelligent Information & Engineering Systems: Proceedings of the 22nd International Conference, KES-2018, Belgrade, Serbia
- Strumbelj E, Kononenko I (2010) An efficient explanation of individual classifications using game theory. *J Mach Learn Res* 11:1–18. Publisher: JMLR. org
- Sundararajan M, Najmi A, III HD, Singh A (2020) The many Shapley values for model explanation. In: III HD, Singh A (eds) *Interna-*

- tional conference on machine learning, Vol. 119 of Proceedings of Machine Learning Research. PMLR, pp 9269–9278
- Sundararajan M, Taly A, Yan Q, Precup D, Teh YW (2017) Axiomatic attribution for deep networks. in: Precup D, Teh YW (eds) Proceedings of the 34th international conference on machine learning, Vol. 70 of Proceedings of Machine Learning Research. PMLR, pp 3319–3328
- Sun M, Geller MR (2018) Efficient characterization of correlated SPAM errors. [arXiv:1810.10523](https://arxiv.org/abs/1810.10523)
- Tannu SS, Qureshi MK (2018) A case for variability-aware policies for nisy-era quantum computers. [arXiv:1805.10224](https://arxiv.org/abs/1805.10224)
- Tian J, Yang W (2024) Explainable quantum neural networks: example-based and feature-based methods. *Electronics* 13. <https://www.mdpi.com/2079-9292/13/20/4136>
- Tucci RR (2005) An introduction to Cartan's KAK decomposition for QC programmers. <https://arxiv.org/abs/quant-ph/0507171>
- Van den Broeck G, Lykov A, Schleich M, Suci D (2022) On the tractability of SHAP explanations. *J Artif Intell Res* 74:851–886
- Wack A, et al (2021) Quality, speed, and scale: three key attributes to measure the performance of near-term quantum computers. [arXiv:2110.14108](https://arxiv.org/abs/2110.14108)
- Weigold M, Barzen J, Leymann F, Salm M, Wirfs-Brock R (2020) Data encoding patterns for quantum computing. In: Wirfs-Brock R (ed) Proceedings of the 27th conference on pattern languages of programs, PLoP '20. The Hillside Group, USA, pp 1–11
- Wilson E, Singh S, Mueller F, Müller HA, Byrd G, Culhane C, DeBenedictis E, Humble T (2020) Just-in-time quantum circuit transpilation reduces noise. In: Müller HA, Byrd G, Culhane C, DeBenedictis E, Humble T (eds) 2020 IEEE international conference on quantum computing and engineering (QCE), pp 345–355
- Winter E (2002) The Shapley value. In: Aumann R, Hart S (eds) Vol. 3 of Handbook of game theory with economic applications. Elsevier, pp 2025–2054. <https://www.sciencedirect.com/science/article/pii/S1574000502030163>
- Xiao H, Wang Z, Zhu Z, Zhou J, Lu J (2022) Shapley-NAS: discovering operation contribution for neural architecture search. [arXiv:2206.09811](https://arxiv.org/abs/2206.09811)
- Zambrano L, Pereira L, Niklitschek S, Delgado A (2020) Estimation of pure quantum states in high dimension at the limit of quantum accuracy through complex optimization and statistical inference. *Sci Rep* 10:12781. <https://doi.org/10.1038/s41598-020-69646-z>
- Zhang Q-S, Zhu S-C (2018) Visual interpretability for deep learning: a survey. *Front Inf Technol Electron Eng* 19:27–39. <https://doi.org/10.1631/FITEE.1700808>
- Zhang V, Nation PD (2023) Characterizing quantum processors using discrete time crystals. [arXiv:2301.07625](https://arxiv.org/abs/2301.07625)
- Zhou Y, Stoudenmire EM, Waintal X (2020) What limits the simulation of quantum computers? *Phys Rev X* 10:041038. <https://doi.org/10.1103/PhysRevX.10.041038>
- Zoufal C, Lucchi A, Woerner S (2019) Quantum generative adversarial networks for learning and loading random distributions. *NPJ Quantum Inf* 5:103. <https://doi.org/10.1038/s41534-019-0223-2>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.